

Chapter 4. Localization and Mapping



Part 2. Localization and Mapping

Contents

4.5.1 Concepts

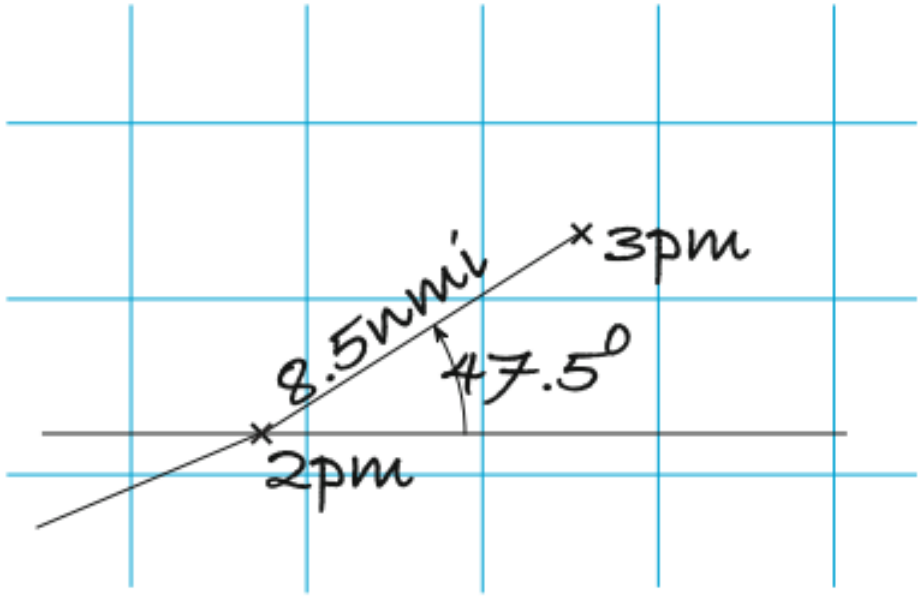
4.5.2 Dead Reckoning Using Odometry

4.5.3 Landmark Maps

DRAFT

4.5.1 Concepts

- In this section, we discuss some of the common techniques used to estimate the pose of a robot in the world – a process known as localization.
- We will introduce the classical navigation principles such as dead reckoning and the use of landmarks on which modern robotic navigation is founded.
- Dead reckoning is the estimation of position based on estimated speed, direction, and time of travel with respect to a previous estimate.

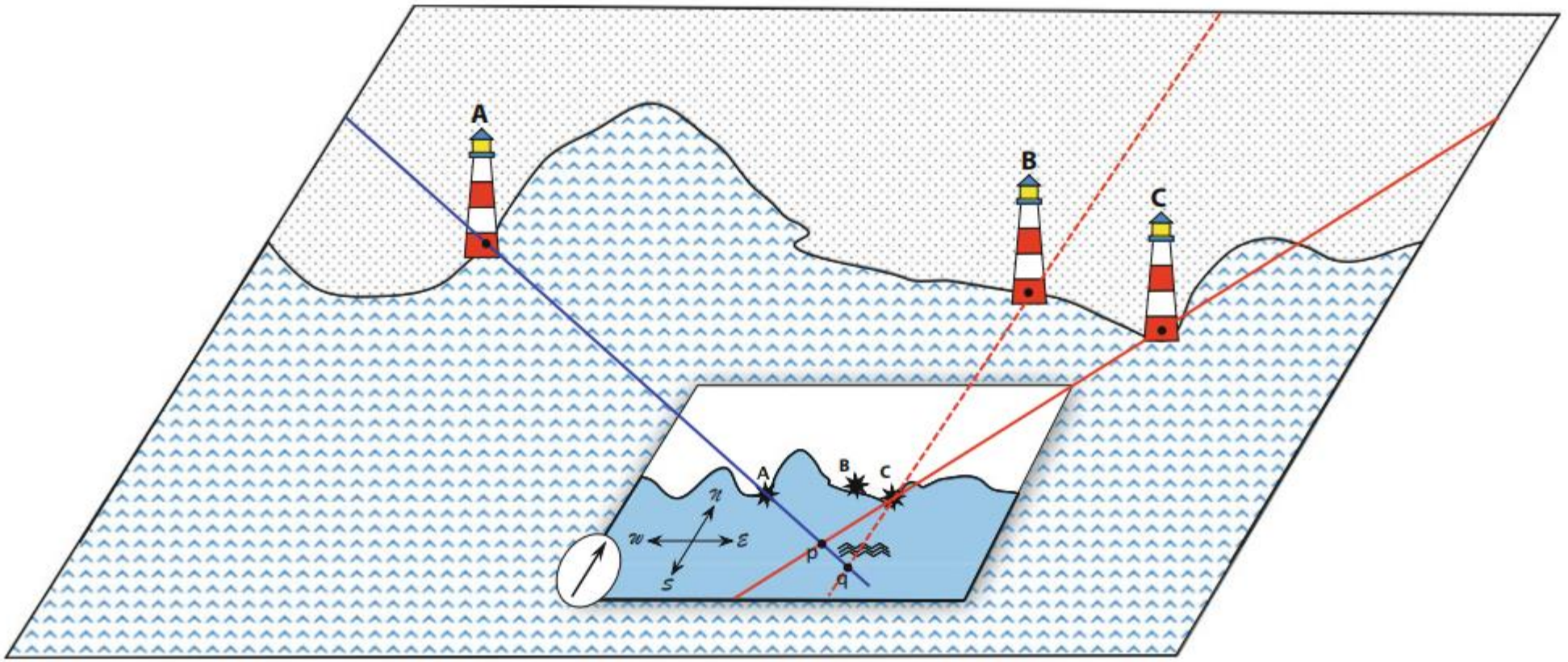


DRAFT

- However, the measurements on which the update is based are subject to both systematic and random error.
- The recursive nature of the process, each estimate is based on the previous one, means that errors will accumulate over time.
- The Phoenicians navigated with crude dead reckoning but wherever possible they used additional information to correct their position estimate – sightings of islands and headlands, primitive maps, and observations of the Sun and the Pole Star.
- A landmark is a visible feature in the environment whose position is known with respect to some coordinate frame.

DRAFT

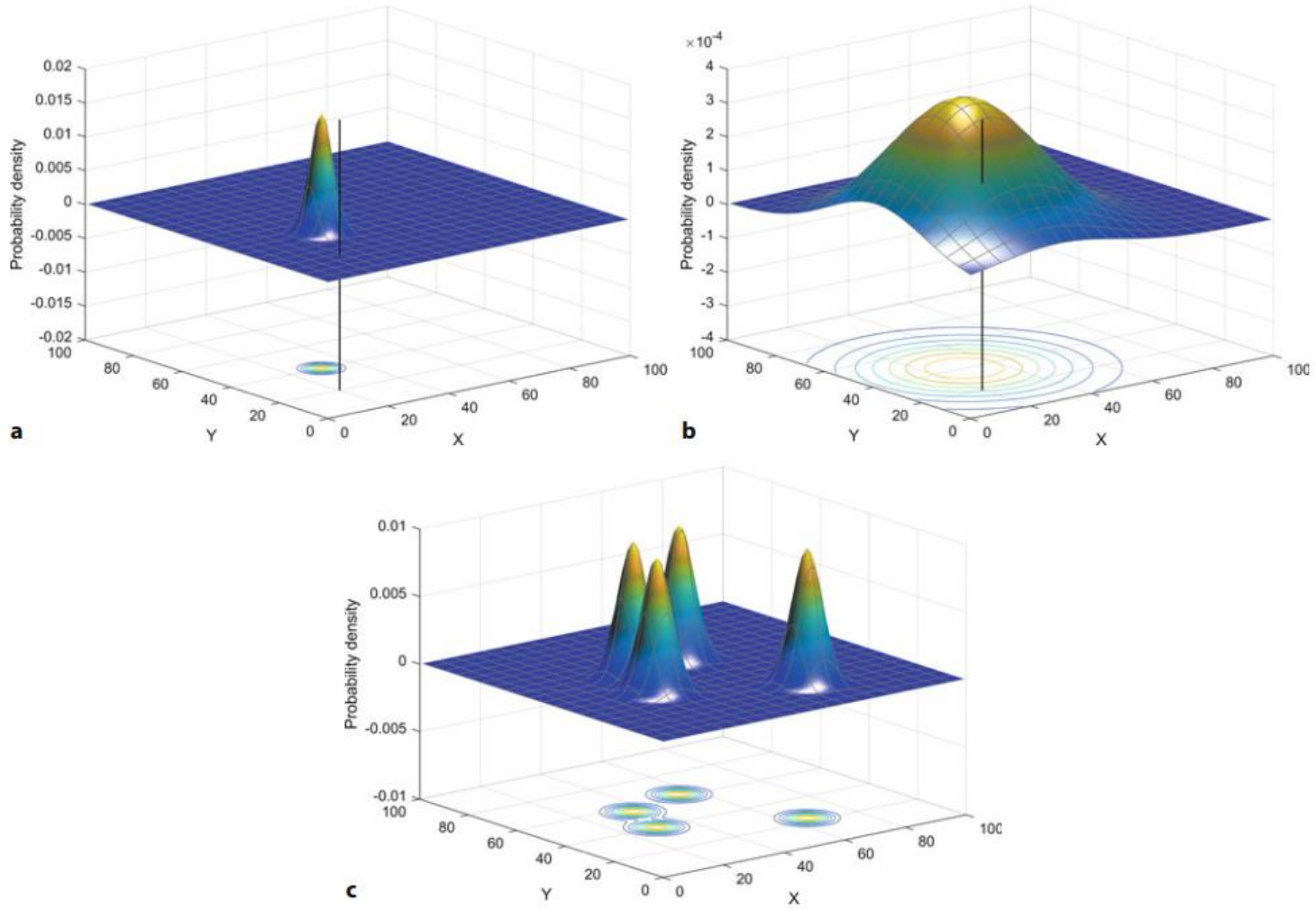
- Resectioning is the estimation of position by measuring the bearing angles to known landmarks.
- Triangulation is the estimation of position by measuring the bearing angles to an unknown point from each of the landmarks.



- A robot's estimate of distance traveled will be imperfect and it may have no map, or perhaps an imperfect or incomplete map.
- Additional information from observed features in the world is critical to minimizing a robot's localization error, but the possibility of data association errors remains.
- We can define the localization problem formally: $\boldsymbol{x}$ is the true, but unknown, position or pose of the robot and $\hat{\boldsymbol{x}}$ is our best estimate of that.
- We also wish to know the uncertainty of the estimate that we can consider in statistical terms as the standard deviation associated with the estimate $\hat{\boldsymbol{x}}$.

DRAFT

- It is useful to describe the robot's estimated position in terms of a probability density function (PDF) over all possible positions of the robot.



- Determining the PDF based on knowledge of how the robot moves and its observations of the world is a problem in estimation, which we can define as:

  the process of inferring the value of some quantity of interest, x, by processing data that is in some way dependent on x.

Probability of being at a pose

The value of a PDF at a point is *not* the probability of being at that position. Instead, consider a small region of the xy-plane, the volume under that region of the PDF is the probability of being in that region.

DRAFT

4.5.2 Dead Reckoning Using Odometry

- Dead reckoning is the estimation of a robot's pose based on its estimated speed, direction, and time of travel with respect to a previous estimate.
- An odometer is a sensor that measures distance traveled.
- In robotics, the term odometry generally means measuring distance and direction of travel.

DRAFT

4.5.2.1 Modeling the Robot

- The first step in estimating the robot's pose is to write a function, $\boldsymbol{f}(\cdot)$, that describes how the robot's configuration changes from one time step to the next.
- We will write a discrete-time model for the change of configuration based on odometry where $\boldsymbol{\delta}\langle k\rangle = (\delta_d, \delta_\theta)^T$ is the distance traveled, δ_d, and change in heading over the preceding interval, δ_θ, and k is the time step.
- The initial pose is represented by an $\mathbf{SE}(2)$ matrix

$$\mathbf{T}\langle k\rangle = \begin{pmatrix} \cos\theta\langle k\rangle & -\sin\theta\langle k\rangle & x\langle k\rangle \\ \sin\theta\langle k\rangle & \cos\theta\langle k\rangle & y\langle k\rangle \\ 0 & 0 & 1 \end{pmatrix}$$

DRAFT

- We choose to move forward in the robot's body frame x-direction by δ_d and then rotate by δ_θ giving the new pose:

$$\mathbf{T}_{\langle k+1\rangle} = \begin{pmatrix} \cos\theta_{\langle k\rangle} & -\sin\theta_{\langle k\rangle} & x_{\langle k\rangle} \\ \sin\theta_{\langle k\rangle} & \cos\theta_{\langle k\rangle} & y_{\langle k\rangle} \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & \delta_d \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \cos\delta_\theta & -\sin\delta_\theta & 0 \\ \sin\delta_\theta & \cos\delta_\theta & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$= \begin{pmatrix} \cos(\theta_{\langle k\rangle} + \delta_\theta) & -\sin(\theta_{\langle k\rangle} + \delta_\theta) & x_{\langle k\rangle} + \delta_d \cos\theta_{\langle k\rangle} \\ \sin(\theta_{\langle k\rangle} + \delta_\theta) & \cos(\theta_{\langle k\rangle} + \delta_\theta) & y_{\langle k\rangle} + \delta_d \sin\theta_{\langle k\rangle} \\ 0 & 0 & 1 \end{pmatrix}$$

which we can represent concisely as a configuration vector $\boldsymbol{q} = (x, y, \theta)$

$$\boldsymbol{q}_{\langle k+1\rangle} = [\mathbf{T}_{\langle k+1\rangle}]_{xy\theta} = \begin{pmatrix} x_{\langle k\rangle} + \delta_d \cos\theta_{\langle k\rangle} \\ y_{\langle k\rangle} + \delta_d \sin\theta_{\langle k\rangle} \\ \theta_{\langle k\rangle} + \delta_\theta \end{pmatrix}$$

Tìm ma trận thuần nhất để tìm ra tọa độ và góc xoay (hướng) cuối cùng của robot.

which gives the new configuration in terms of the previous configuration and the odometry.

DRAFT

- In practice, odometry is not perfect and we model the error by imagining a random number generator that corrupts the output of a perfect odometer.
- The robot's configuration at the next time step, including the odometry error, is

$$\boldsymbol{q}\langle k+1\rangle = \boldsymbol{f}(\boldsymbol{q}\langle k\rangle,\ \boldsymbol{\delta}\langle k\rangle,\ \boldsymbol{v}\langle k\rangle) = \begin{pmatrix} x\langle k\rangle + (\delta_d + v_d)\cos\theta\langle k\rangle \\ y\langle k\rangle + (\delta_d + v_d)\sin\theta\langle k\rangle \\ \theta\langle k\rangle + \delta_\theta + v_\theta \end{pmatrix}$$

- Có cộng thêm sai số)

where k is the time step; $\boldsymbol{\delta}\langle k\rangle = (\delta_d, \delta_\theta)^T$ is the odometry measurement and $\boldsymbol{v}\langle k\rangle = (v_d, v_\theta)^T$ is the random measurement noise over the preceding interval.

- We model the odometry noise as a zero-mean multivariate Gaussian process with covariance

$$\mathbf{V} = \begin{pmatrix} \sigma_d^2 & 0 \\ 0 & \sigma_\theta^2 \end{pmatrix}$$

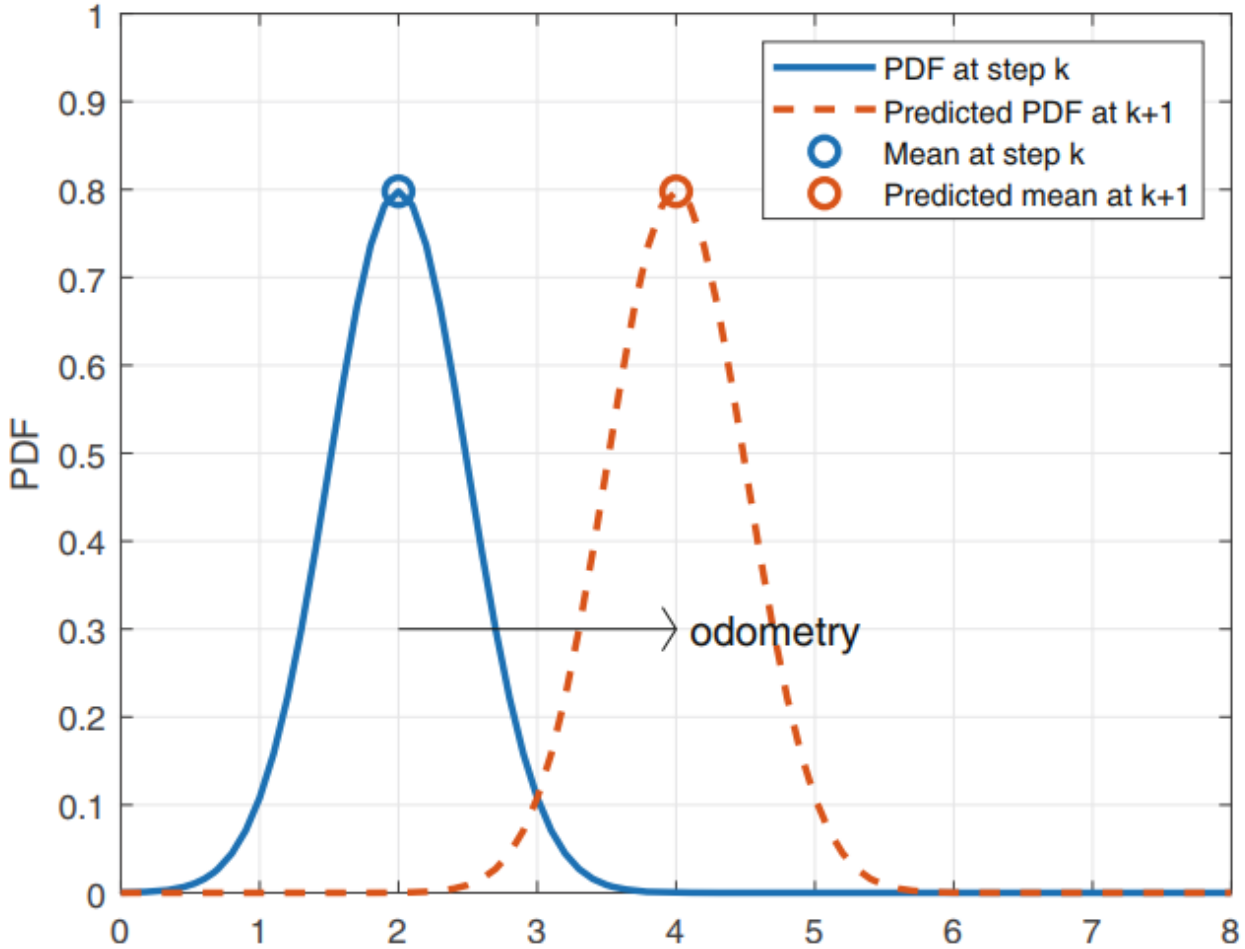
phương sai theo quãng đường
phương sai theo góc.

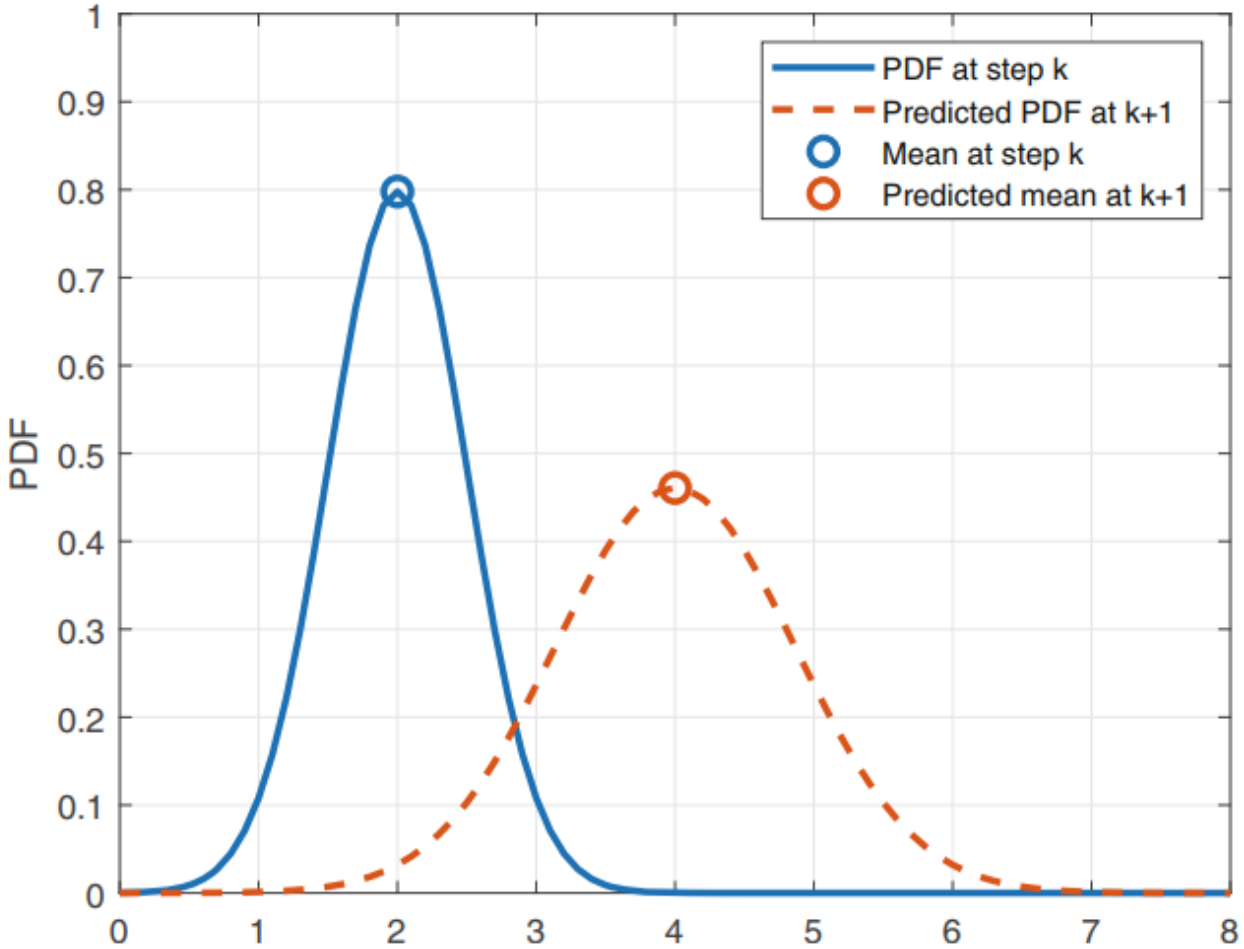
DRAFT

4.5.2.2 Estimating Pose

- The problem we face is how to estimate our new pose given the previous pose and noisy odometry.
- The mathematical tool that we will use to solve this problem is the Kalman filter. This is a recursive algorithm that updates, at each time step, the optimal estimate of the unknown true configuration and the uncertainty associated with that estimate, based on the previous estimate, the control input, and noisy measurement data.
- This filter can be shown to provide the optimal estimate of the system state assuming that the noise is Gaussian and has a zero mean.
- The Kalman filter is formulated for linear systems, but our model of the robot's motion is nonlinear – for this case we use the extended Kalman filter.

DRAFT


PDF at step k
Predicted PDF at k+1
Mean at step k
Predicted mean at k+1
odometry
PDF
0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1
0
1
2
3
4
5
6
7
8


PDF at step k
Predicted PDF at k+1
Mean at step k
Predicted mean at k+1
PDF
0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1
0
1
2
3
4
5
6
7
8

- For the multi-dimensional robot localization problem, the state vector is the robot's configuration

$$\boldsymbol{x} = (x_v,\ y_v,\ \theta_v)^\top$$

and the prediction equations

$$\hat{\boldsymbol{x}}^+_{\langle k+1\rangle} = \boldsymbol{f}(\hat{\boldsymbol{x}}_{\langle k\rangle},\ \hat{\boldsymbol{u}}_{\langle k\rangle})$$

$$\hat{\mathbf{P}}^+_{\langle k+1\rangle} = \mathbf{F}_x\hat{\mathbf{P}}_{\langle k\rangle}\mathbf{F}_x^\top + \mathbf{F}_v\hat{\mathbf{V}}\mathbf{F}_v^\top$$

dấu + là dự đoán

- The term $\hat{\boldsymbol{x}}^+\langle k+1\rangle$ indicates an estimate of x at time k+1 based on information up to, and including, time k.
- $\hat{\boldsymbol{u}}\langle k\rangle$ is the input to the process, which in this case is the measured odometry, so $\hat{\boldsymbol{u}}\langle k\rangle = \boldsymbol{\delta}\langle k\rangle$.
- $\hat{\mathbf{V}}$ is our estimate of the covariance of the odometry noise.

DRAFT

- The covariance matrix is symmetric

$$\hat{\mathbf{P}} = \left(\begin{array}{cc|c} \sigma_x^2 & \mathrm{cov}(x,y) & \mathrm{cov}(x,\theta) \\ \mathrm{cov}(x,y) & \sigma_y^2 & \mathrm{cov}(y,\theta) \\ \hline \mathrm{cov}(x,\theta) & \mathrm{cov}(y,\theta) & \sigma_\theta^2 \end{array}\right)$$

and represents uncertainty in the estimated robot configuration, as well as dependence between elements of the configuration.

- For a function $\boldsymbol{y} = \boldsymbol{f}(\boldsymbol{x})$ the covariance of $\mathbf{x}$ and $\mathbf{y}$ are related by $\boldsymbol{\Sigma_y} = \mathbf{F}\boldsymbol{\Sigma_x}\mathbf{F}^{\mathrm{T}}$ where F is a Jacobian matrix

$$\mathbf{F}_x = \left.\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}\right|_{v=0} = \begin{pmatrix} 1 & 0 & -\delta_d \sin\theta_v \\ 0 & 1 & \delta_d \cos\theta_v \\ 0 & 0 & 1 \end{pmatrix}$$

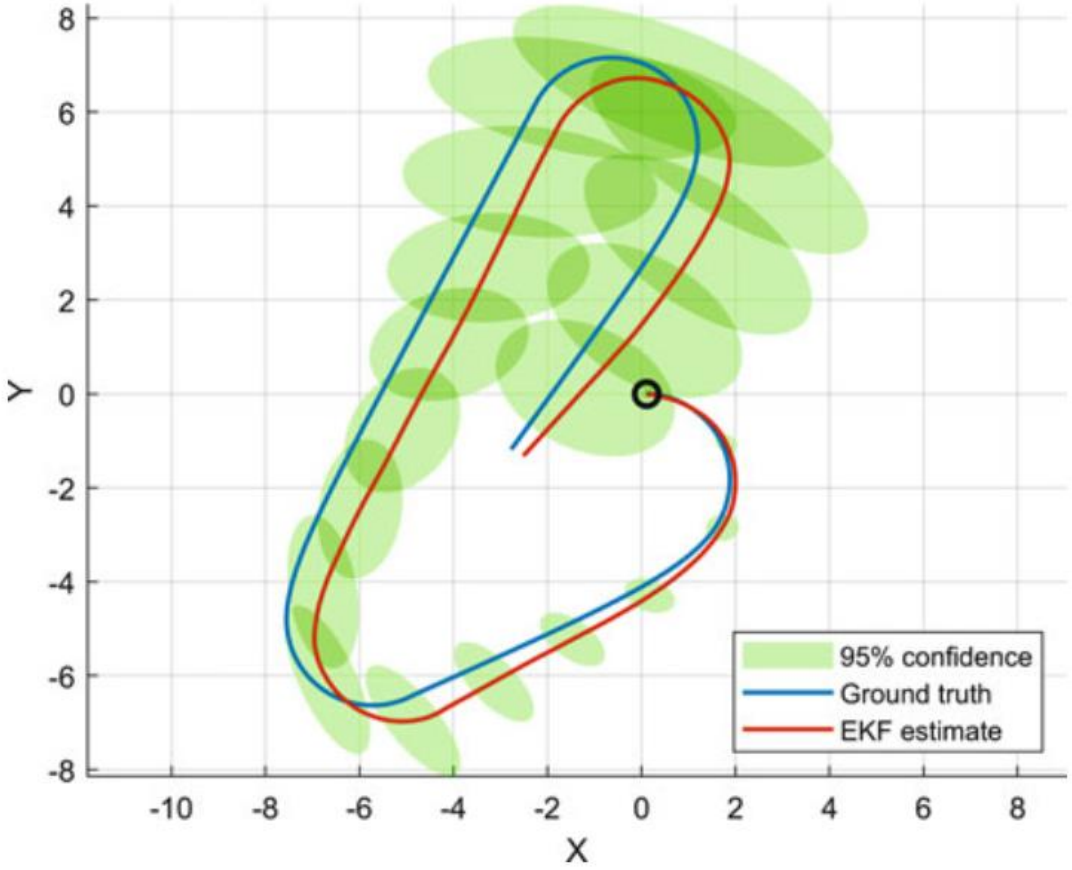
$$\mathbf{F}_v = \left.\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{v}}\right|_{v=0} = \begin{pmatrix} \cos\theta_v & 0 \\ \sin\theta_v & 0 \\ 0 & 1 \end{pmatrix}$$

DRAFT

- To simulate the robot and the EKF using the RVC Toolbox, we must define the initial covariance. We choose a diagonal matrix

```
>> P0 = diag([0.05 0.05 deg2rad(0.5)].^2);
```

- Running the filter for 400 time steps drives the robot along a random path starting at the origin.



elip càng lớn thì độ ko chắc chắn càng lớn. Phương sai càng lớn.

- We see some divergence between the true and estimated robot path.
- The covariance at time step 150 is

```
>> P150 = ekf.history(150).P
P150 =
    0.0868    0.0163   -0.0070
    0.0163    0.2676    0.0399
   -0.0070    0.0399    0.0115
```

- The standard deviation of the PDF associated with the robot's x-coordinate is

```
>> sqrt(P150(1,1))
ans =
    0.2946
```

kỳ vọng 95% giá trị đúng rơi vào vùng này

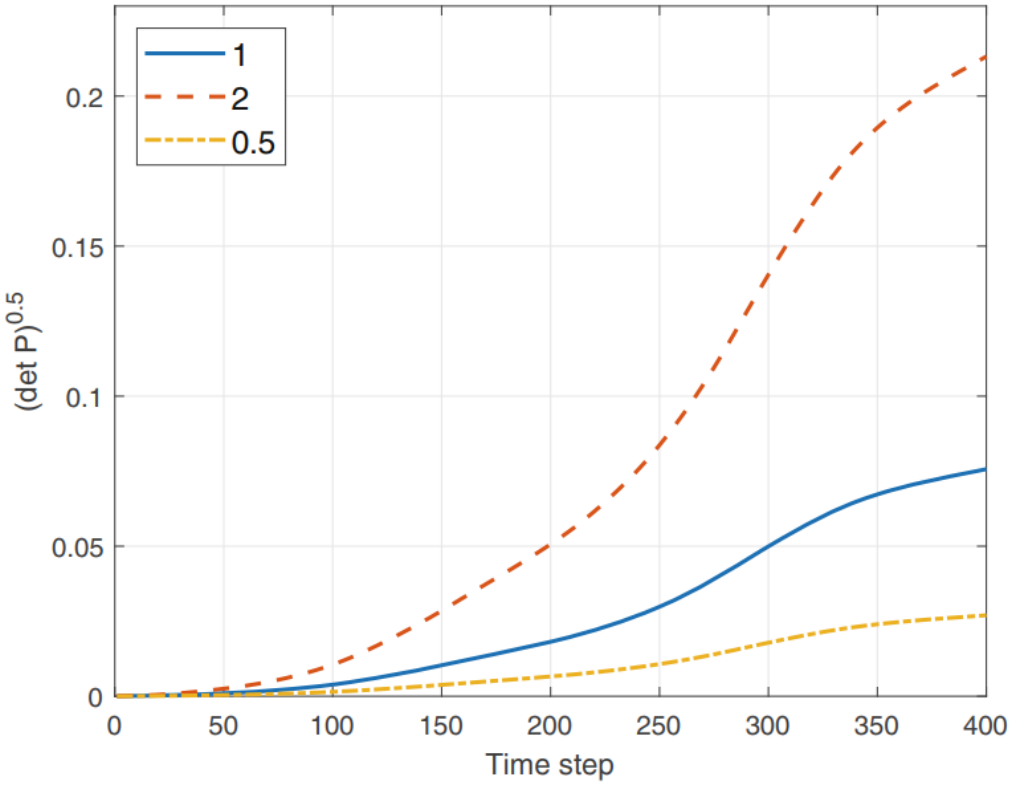
- There is a 95% chance that the robot's x-coordinate is within the $\pm 2\sigma$ bound or ± 0.59m in this case.

DRAFT

- The off-diagonal terms are correlation coefficients and indicate that the uncertainties between the corresponding variables are related.
- For example, the value $p_{1,3} = p_{3,1} = -0.1253$ indicates that the uncertainties in x and are related – error in heading angle causes error in x-position and vice versa. Conversely, new information about can be used to correct as well as x.
- The uncertainty in position is described by the top-left 2x2 covariance submatrix of $\widehat{\mathbf{P}}$, which can be interpreted as an ellipse defining a confidence bound (95%) on position.

DRAFT

- The estimated uncertainty in position and heading is given by $\sqrt{\det(\hat{\mathbf{P}})}$, which we can plot as a function of time.
- We observe that the uncertainty never decreases, and this is because in the covariance prediction equation we keep adding a positive-definite matrix (the second term) to the estimated covariance.



DRAFT

V vs $\hat{\mathbf{V}}$

We have used the odometry covariance matrix `V` twice. The first usage, in the `BicycleVehicle` constructor, is the covariance $\mathbf{V}$ of the zero-mean Gaussian noise source that is added to the true odometry to *simulate* odometry error in (6.2). In a real application this noise is generated by some physical process *hidden inside* the robot and we would not know its parameters. The second usage, in the `EKF` constructor is $\hat{\mathbf{V}}$, which is our best *estimate* of the odometry covariance and is used in the filter's state covariance update equation (6.4).

The relative values of $\mathbf{V}$ and $\hat{\mathbf{V}}$ control the rate of uncertainty growth as shown in Fig. 6.6. If $\hat{\mathbf{V}} > \mathbf{V}$ then $\hat{\mathbf{P}}$ will be larger than it should be and the filter is pessimistic – it overestimates uncertainty and is less certain than it should be. If $\hat{\mathbf{V}} < \mathbf{V}$ then $\hat{\mathbf{P}}$ will be smaller than it should be and the filter will be *overconfident* of its estimate – the actual uncertainty is greater than the estimated uncertainty. In practice some experimentation is required to determine the appropriate value for the estimated covariance.

Excurse: Error Ellipse

We consider the PDF of the robot's position (ignoring orientation) such as shown in Fig. 6.3 to be a 2-dimensional Gaussian probability density function

$$p(\boldsymbol{x}) = \frac{1}{(2\pi)\det(\mathbf{P}_{xy})^{1/2}} e^{\left\{-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu}_x)^\top \mathbf{P}_{xy}^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_x)\right\}}$$

a

where $\boldsymbol{x} = (x, y)^\top$ is the position of the robot, $\boldsymbol{\mu}_x = (\hat{x}, \hat{y})^\top$ is the estimated mean position and $\mathbf{P}_{xy} \in \mathbb{R}^{2\times 2}$ is the position covariance matrix, the top left of the covariance matrix $\mathbf{P}$ as shown in (6.5). A horizontal cross-section of the PDF is a contour of constant probability which is an ellipse defined by the points $\boldsymbol{x}$ such that

$$(\boldsymbol{x}-\boldsymbol{\mu}_x)^\top \mathbf{P}_{xy}^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_x) = s \ .$$

Such error ellipses are often used to represent positional uncertainty as shown in Fig. 6.5. A large ellipse corresponds to a wider PDF peak and less certainty about position. To obtain a particular confidence contour (for example, 99%) we choose s as the inverse of the χ^2 cumulative distribution function for 2 degrees of freedom. In MATLAB®, we can use the function `chi2inv(C,2)`, where `C` $\in [0, 1]$ is the confidence value. Such confidence values can be passed to several `EKF` methods when specifying error ellipses.

A handy scalar measure of total position uncertainty is the area of the ellipse $\pi r_1 r_2$ where the radii $r_i = \sqrt{\lambda_i}$ and λ_i are the eigenvalues of $\mathbf{P}_{xy}$. Since $\det(\mathbf{P}_{xy}) = \pi\lambda_i$, the ellipse area – the scalar uncertainty – is proportional to $\sqrt{\det(\mathbf{P}_{xy})}$. See also ▶ App. C.1.4 and G.

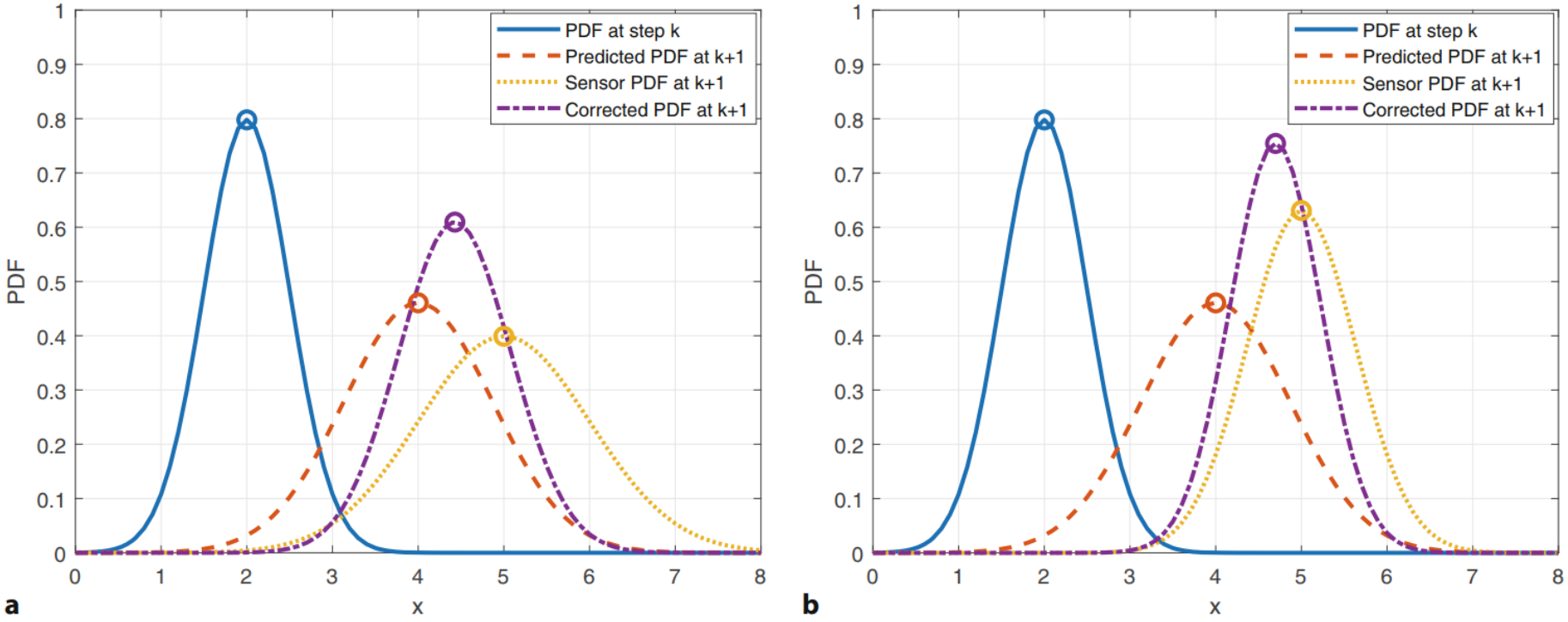
4.5.3 Landmark Maps

- One common way to represent the robot's environment is to keep track of the positions and covariances of distinct landmarks, which are salient and recognizable features.
- In indoor environments, landmarks may be doors and hallway corners; in outdoor environments, tree trunks, or intersections.
- When dealing with landmarks, we typically assume that we have a sensor that can directly measure the landmark's position, or the landmark position can be determined through sensor processing.
- This section focuses on how to solve the localization, mapping, and SLAM problems for landmark maps.

DRAFT

4.5.3.1 Localizing in a Landmark Map

- We have seen how uncertainty in position grows without bound using dead reckoning alone.
- The solution is to use additional information from observations of known features, or landmarks, in the world.



- For a robotics problem we consider that the robot is able to observe a set of landmarks using an onboard sensor.
- We will consider that the position of the landmarks is known, and we will use a map that contains N fixed but randomly located landmarks whose positions are known.
- The robot is equipped with a sensor that provides observations of the landmarks with respect to the robot as described by

$$\boldsymbol{z = h(q,\ p_i)}$$

where $\boldsymbol{q} = (x_v, y_v, \theta_v)^T$ is the robot configuration, and $\boldsymbol{p_i} = (x_i, y_i)^T$ is the known position of the ith landmark in the world frame.

DRAFT

- To make this tangible, we will consider a common type of sensor that measures the range and bearing angle to a landmark in the environment, for instance a radar or lidar.
- The sensor is mounted onboard the robot so the observation of the ith landmark is

$$\boldsymbol{z} = \boldsymbol{h}(\boldsymbol{q},\ \boldsymbol{p}_i) = \begin{pmatrix} \sqrt{(y_i - y_v)^2 + (x_i - x_v)^2} \\ \tan^{-1} \frac{y_i - y_v}{x_i - x_v} - \theta_v \end{pmatrix} + \begin{pmatrix} w_r \\ w_\beta \end{pmatrix}$$

where $\boldsymbol{z} = (r, \beta)^T$ and r is the range, β - the bearing angle; and $\boldsymbol{w} = \left(w_r, w_\beta\right)^T$ is a zero-mean Gaussian random variable that models errors in the sensor.

$$\begin{pmatrix} w_r \\ w_\beta \end{pmatrix} \sim N(0,\ \mathbf{W}), \quad \mathbf{W} = \begin{pmatrix} \sigma_r^2 & 0 \\ 0 & \sigma_\beta^2 \end{pmatrix}$$

DRAFT

- The robot can estimate the range and bearing angle to the landmark based on its own estimated position and the known position of the landmark from the map.
- The difference between the observation z and the estimated observation

$$\boldsymbol{v} = \boldsymbol{z}\langle k\rangle - \boldsymbol{h}\left(\hat{\boldsymbol{x}}^{+}\langle k\rangle, \boldsymbol{p}_i\right)$$

indicates an error in the robot's pose estimate $\hat{\boldsymbol{x}}^{+}$ – the robot isn't where it thought it was. This difference is called the innovation, since it represents valuable new information, and is key to the operation of the Kalman filter.

- The second step of the Kalman filter updates the predicted state and covariance

$$\hat{\boldsymbol{x}}\langle k+1\rangle = \hat{\boldsymbol{x}}^{+}\langle k+1\rangle + \mathbf{K}\boldsymbol{v}\langle k+1\rangle$$

uses a gain term K to add some amount of the innovation to the predicted state estimate so as to reduce the estimation error.

DRAFT

- The Kalman Gain is given by

$$\mathbf{K} = \mathbf{P}^{+}{}_{\langle k+1\rangle}\mathbf{H}_x^{\top}\left(\mathbf{H}_x\mathbf{P}^{+}{}_{\langle k+1\rangle}\mathbf{H}_x^{\top} + \mathbf{H}_w\hat{\mathbf{W}}\mathbf{H}_w^{\top}\right)^{-1}$$

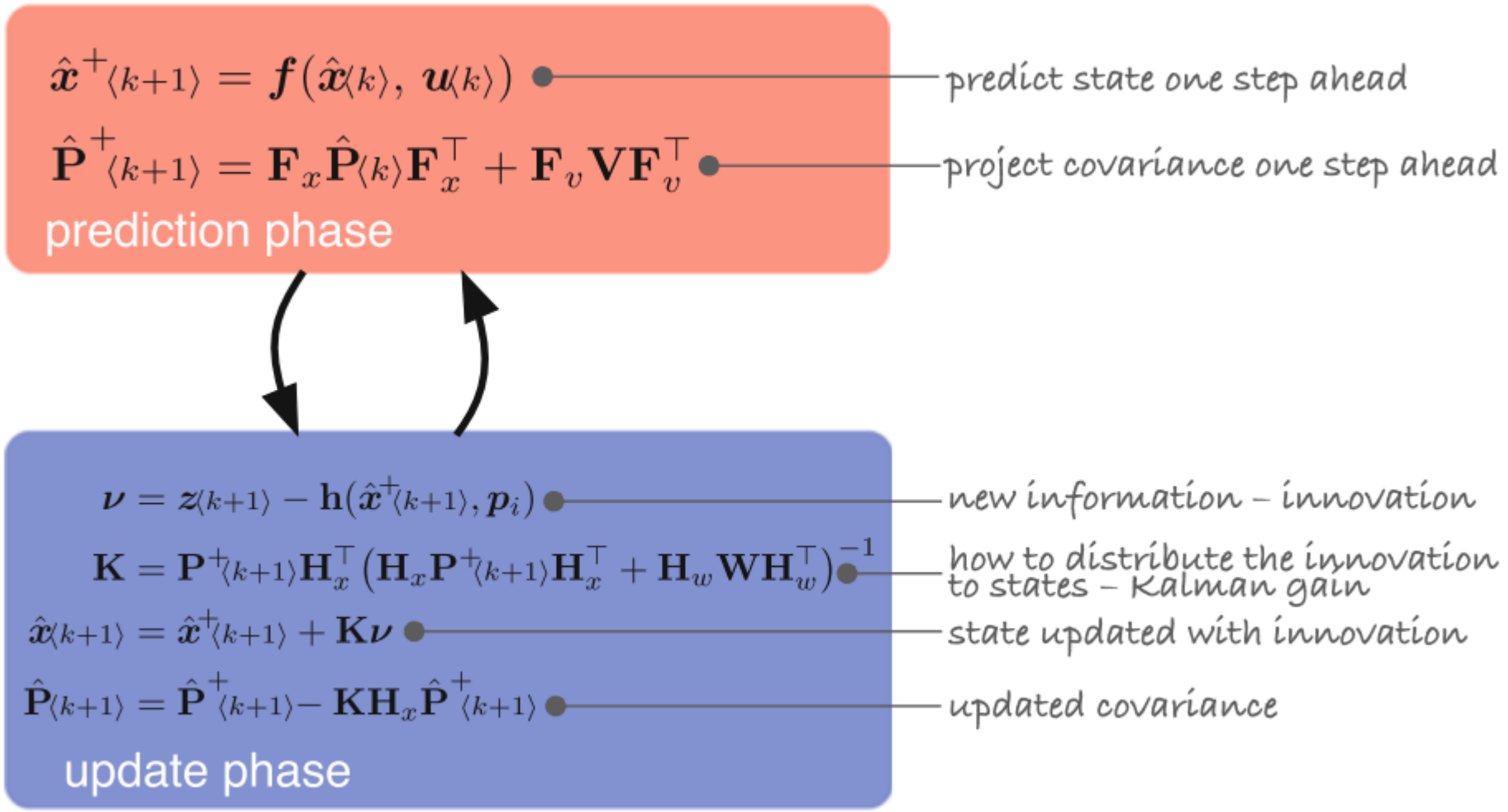
where $\hat{\mathbf{W}}$ is the estimated covariance of the sensor noise and $\mathbf{H}_x$ and $\mathbf{H}_w$ are Jacobians.

$$\mathbf{H}_x = \left.\frac{\partial \boldsymbol{h}}{\partial \boldsymbol{x}}\right|_{\boldsymbol{w}=0} = \begin{pmatrix} -\frac{x_i - x_v}{r} & -\frac{y_i - y_v}{r} & 0 \\ \frac{y_i - y_v}{r^2} & -\frac{x_i - x_v}{r^2} & -1 \end{pmatrix} \qquad \mathbf{H}_w = \left.\frac{\partial \boldsymbol{h}}{\partial \boldsymbol{w}}\right|_{\boldsymbol{w}=0} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

- The covariance update also depends on the Kalman gain

$$\hat{\mathbf{P}}_{\langle k+1\rangle} = \hat{\mathbf{P}}^{+}{}_{\langle k+1\rangle} - \mathbf{K}\mathbf{H}_x\hat{\mathbf{P}}^{+}{}_{\langle k+1\rangle}$$

DRAFT


$\hat{\boldsymbol{x}}^{+}_{\langle k+1\rangle} = \boldsymbol{f}(\hat{\boldsymbol{x}}_{\langle k\rangle}, \boldsymbol{u}_{\langle k\rangle})$
$\hat{\mathbf{P}}^{+}_{\langle k+1\rangle} = \mathbf{F}_x \hat{\mathbf{P}}_{\langle k\rangle} \mathbf{F}_x^{\top} + \mathbf{F}_v \mathbf{V} \mathbf{F}_v^{\top}$
prediction phase
predict state one step ahead
project covariance one step ahead
$\boldsymbol{\nu} = \boldsymbol{z}_{\langle k+1\rangle} - \mathbf{h}(\hat{\boldsymbol{x}}^{+}_{\langle k+1\rangle}, \boldsymbol{p}_i)$
$\mathbf{K} = \mathbf{P}^{+}_{\langle k+1\rangle} \mathbf{H}_x^{\top} \left(\mathbf{H}_x \mathbf{P}^{+}_{\langle k+1\rangle} \mathbf{H}_x^{\top} + \mathbf{H}_w \mathbf{W} \mathbf{H}_w^{\top} \right)^{-1}$
$\hat{\boldsymbol{x}}_{\langle k+1\rangle} = \hat{\boldsymbol{x}}^{+}_{\langle k+1\rangle} + \mathbf{K}\boldsymbol{\nu}$
$\hat{\mathbf{P}}_{\langle k+1\rangle} = \hat{\mathbf{P}}^{+}_{\langle k+1\rangle} - \mathbf{K}\mathbf{H}_x \hat{\mathbf{P}}^{+}_{\langle k+1\rangle}$
update phase
new information – innovation
how to distribute the innovation to states – Kalman gain
state updated with innovation
updated covariance

- The Kalman Gain is given by

$$\mathbf{K} = \mathbf{P}^{+}{}_{\langle k+1\rangle}\mathbf{H}_x^{\top}\left(\mathbf{H}_x\mathbf{P}^{+}{}_{\langle k+1\rangle}\mathbf{H}_x^{\top} + \mathbf{H}_w\hat{\mathbf{W}}\mathbf{H}_w^{\top}\right)^{-1}$$

where $\hat{\mathbf{W}}$ is the estimated covariance of the sensor noise and $\mathbf{H}_x$ and $\mathbf{H}_w$ are Jacobians.

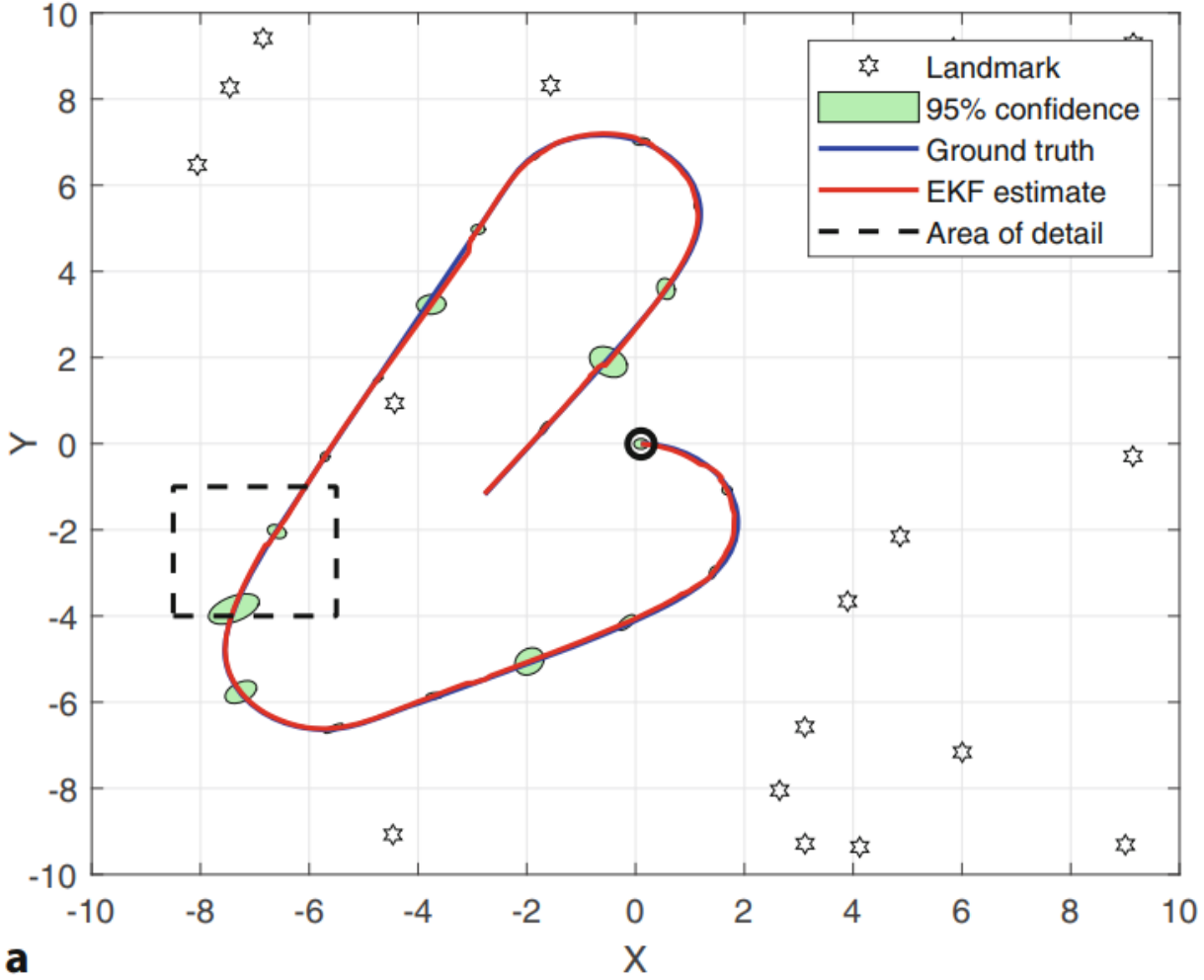
$$\mathbf{H}_x = \left.\frac{\partial \boldsymbol{h}}{\partial \boldsymbol{x}}\right|_{\boldsymbol{w}=0} = \begin{pmatrix} -\frac{x_i - x_v}{r} & -\frac{y_i - y_v}{r} & 0 \\ \frac{y_i - y_v}{r^2} & -\frac{x_i - x_v}{r^2} & -1 \end{pmatrix} \qquad \mathbf{H}_w = \left.\frac{\partial \boldsymbol{h}}{\partial \boldsymbol{w}}\right|_{\boldsymbol{w}=0} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

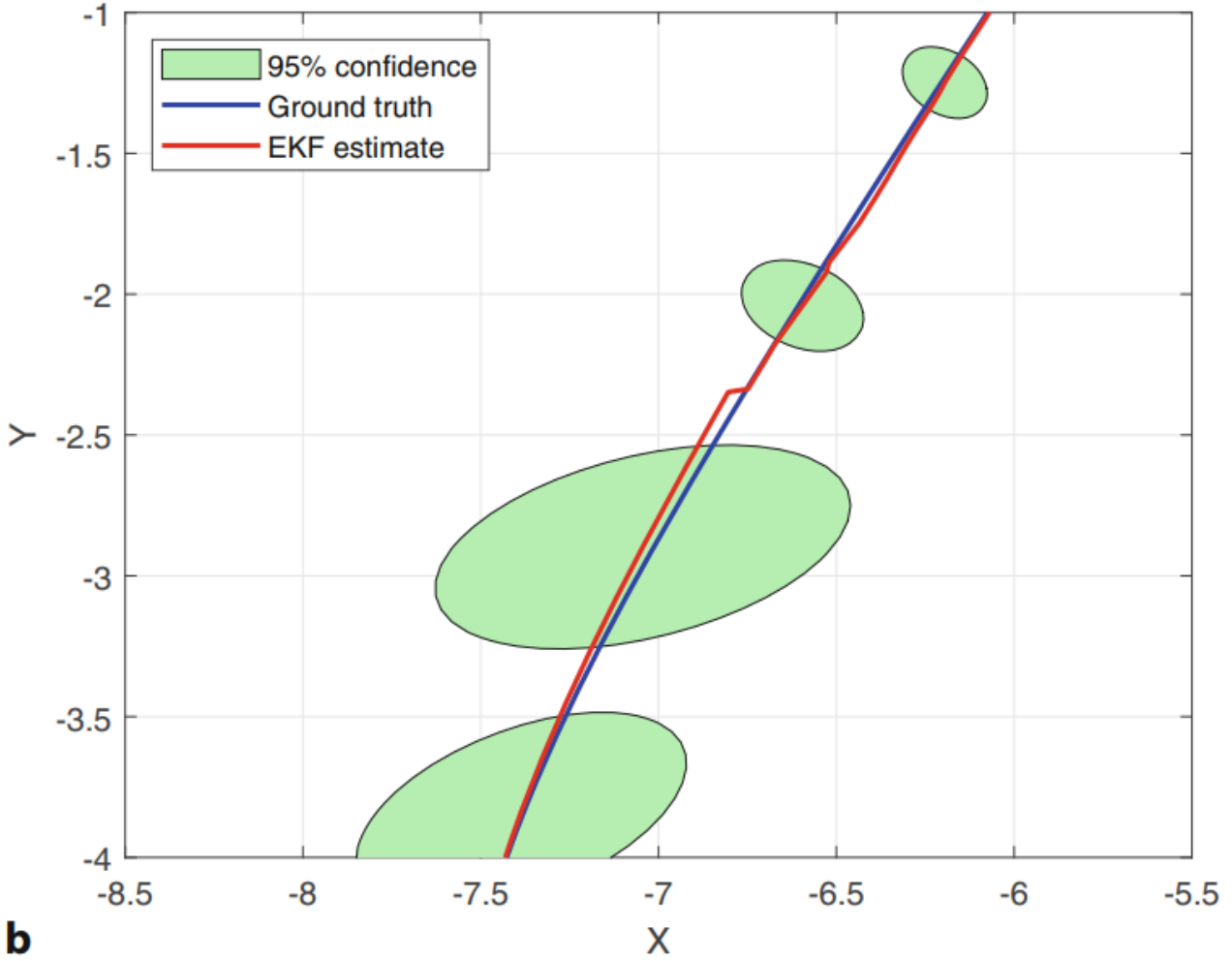
- The covariance update also depends on the Kalman gain

$$\hat{\mathbf{P}}_{\langle k+1\rangle} = \hat{\mathbf{P}}^{+}{}_{\langle k+1\rangle} - \mathbf{K}\mathbf{H}_x\hat{\mathbf{P}}^{+}{}_{\langle k+1\rangle}$$

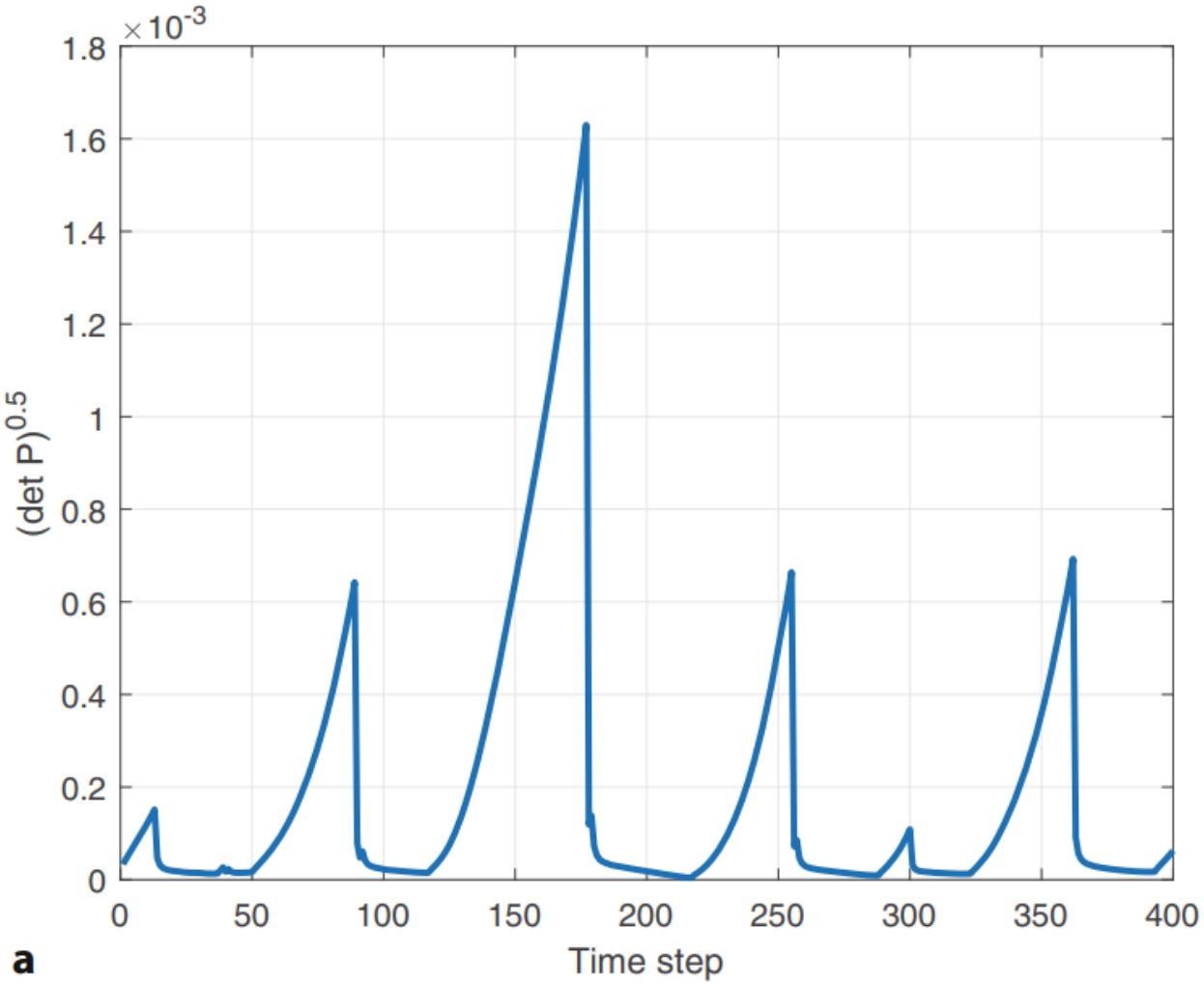
DRAFT

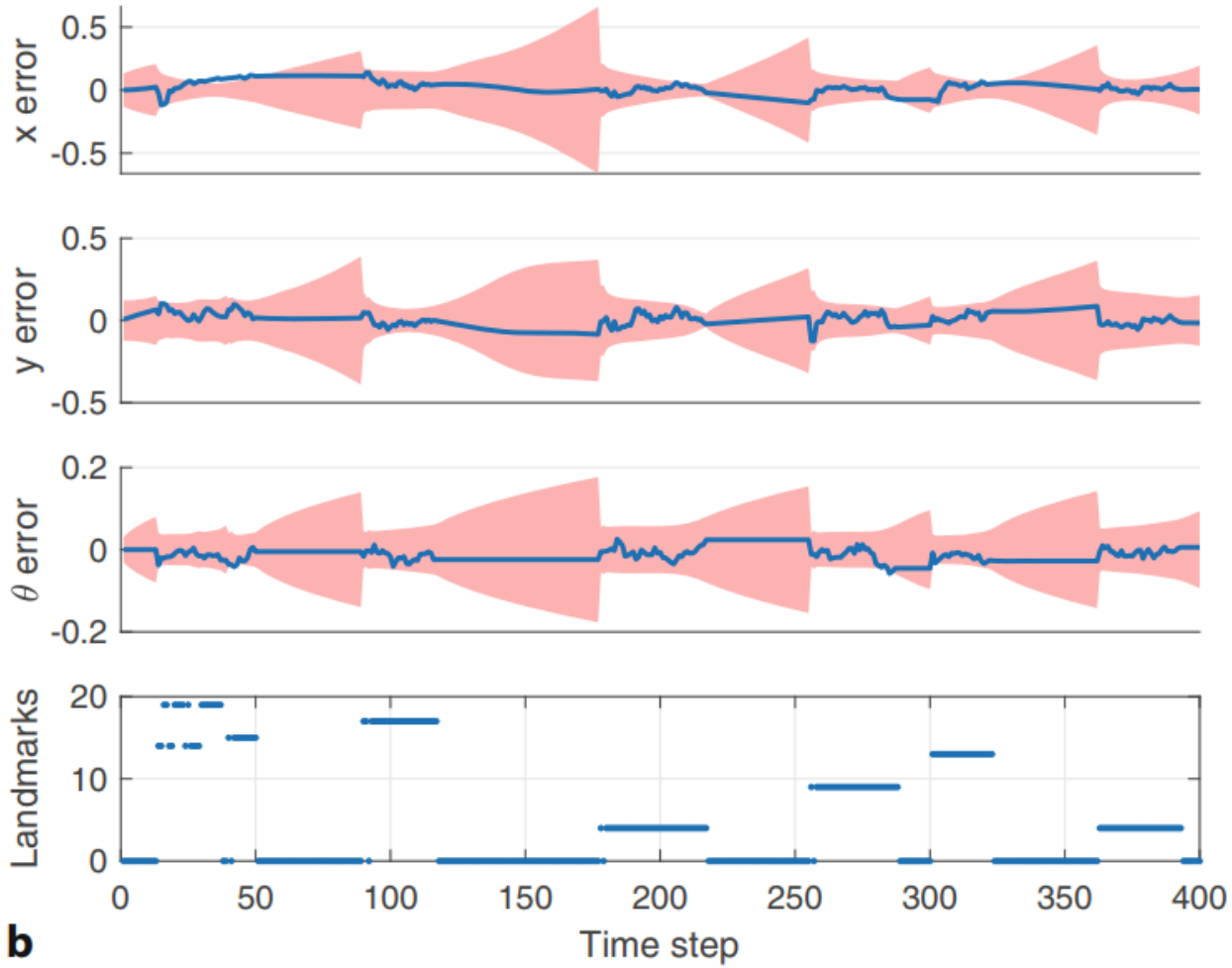
- We see that the error ellipses grow as the robot moves, just as we observed for the dead reckoning case and then shrink due to a landmark observation that corrects the robot's estimated state.





- When the robot observes a landmark, the estimated covariance is dramatically reduced.
- The error is mostly within the 95% confidence bound.





- The EKF is an extremely powerful tool that allows data from many and varied sensors to update the state, which is why the estimation problem is also referred to as sensor fusion.
- For example, heading angle from a compass, yaw rate from a gyroscope, target bearing angle from a camera, and position from GPS could all be used to update the state.

DRAFT

Excurse: Data Association

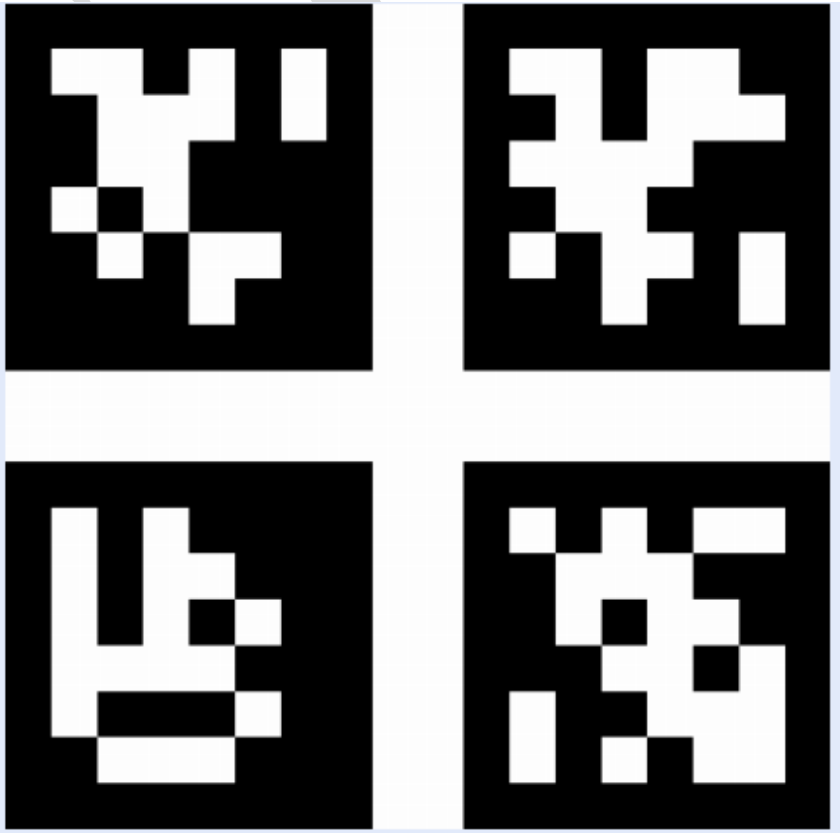
So far, we have assumed that we can determine the identity of the observed landmark, but in reality this is rarely the case. Instead, we can compare our observation to the predicted position of all currently known landmarks and decide which landmark it is most likely to be, or whether it is a new landmark. This decision needs to consider the uncertainty associated with the robot's pose, the sensor measurement, and the landmarks in the map. This is the data association problem. Errors in this step are potentially catastrophic – incorrect innovation would introduce error in the state of the robot that increases the chance of an incorrect data association on the next cycle. In practice, filters only use a landmark when there is a very high confidence in its estimated identity – a process that involves Mahalanobis distance and χ^2 confidence tests. If the situation is ambiguous, it is best not to use the landmark – a bad update can do more harm than good.

An alternative is to use a multi-hypothesis estimator, such as the particle filter that we will discuss in ▶ Sect. 6.2.4, which can model the possibility of observing landmark A or landmark B, and future observations will reinforce one hypothesis and weaken the others. The Extended Kalman filter uses a Gaussian probability model, with just one peak (or mode), which limits it to holding only one hypothesis about the robot's pose. (Picture: the wreck of the Tararua, 1881)



Excurse: Artifcial Landmarks

For some applications it might be appropriate to use artificial landmarks, or fiducial markers, that can be cheaply sensed by a camera. These are not only visually distinctive in the environment, but they can also encode a unique identity. Commonly used 2-dimensional bar codes include ArUco markers, ARTags, and AprilTags (shown here). If the camera is calibrated (see ▶ Sect. 13.2), we can determine the range to the marker and its surface normal. MATLAB supports reading AprilTags with the `readAprilTag` function and its usage is discussed in ▶ Sect. 13.6.1.



4.5.3.2 Creating a Landmark Map

- In this section, we consider the problem of creating a map – determining the positions of the landmarks – in an environment that the robot is moving through.
- As before, we have a range and bearing sensor mounted on the robot, which imperfectly measures the position of landmarks with respect to the robot.
- We will assume that the sensor can determine the identity of each observed landmark, and that the robot's pose is known perfectly – it has ideal localization. This is unrealistic but this scenario is an important conceptual stepping stone to the next section.

DRAFT

- Since the robot's pose is known perfectly, we do not need to estimate it, but we do need to estimate the coordinates of the landmarks.
- For this problem the state vector comprises the coordinates of the M landmarks that have been observed so far

$$\boldsymbol{x} = (x_1,\ y_1,\ x_2,\ y_2,\ \ldots\ x_M,\ y_M)^\top \in \mathbb{R}^{2M\times 1}$$

- The covariance matrix is symmetric

$$\hat{\mathbf{P}} = \left(\begin{array}{cc|cc|c} \sigma_{x_1}^2 & \mathrm{cov}(x_1,y_1) & \mathrm{cov}(x_1,x_2) & \mathrm{cov}(x_1,y_2) & \cdots \\ \mathrm{cov}(x_1,y_1) & \sigma_{y_1}^2 & \mathrm{cov}(y_1,x_2) & \mathrm{cov}(y_1,y_2) & \cdots \\ \hline \mathrm{cov}(x_1,x_2) & \mathrm{cov}(x_1,y_2) & \sigma_{x_2}^2 & \mathrm{cov}(x_2,y_2) & \cdots \\ \mathrm{cov}(y_1,x_2) & \mathrm{cov}(y_1,y_2) & \mathrm{cov}(x_2,y_2) & \sigma_{y_2}^2 & \\ \hline \vdots & \vdots & \vdots & \vdots & \ddots \end{array}\right)$$

and each 2x2 block on the diagonal is the covariance of the corresponding landmark position. The off-diagonal blocks are the covariance between the landmarks.

DRAFT

- Initially $M = 0$ and is incremented every time a previously unseen landmark is observed. The state vector has a variable length since we do not know in advance how many landmarks exist in the environment.
- The estimated covariance $\hat{\mathbf{P}} \in \mathbb{R}^{2M \times 2M}$ also has variable size.
- The prediction equation is straightforward in this case, since the landmarks are assumed to be stationary

$$\hat{\boldsymbol{x}}^{+}_{\langle k+1 \rangle} = \hat{\boldsymbol{x}}_{\langle k \rangle}$$

$$\hat{\mathbf{P}}^{+}_{\langle k+1 \rangle} = \hat{\mathbf{P}}_{\langle k \rangle}$$

DRAFT

- We introduce the function $\boldsymbol{g}(\cdot)$, which is the inverse of $\boldsymbol{h}(\cdot)$ and returns the world coordinates of the observed landmark based on the known robot pose (x_v, y_v, θ_v) and the sensor observation (this is also known as the inverse measurement function or inverse sensor model)

$$\boldsymbol{g}(\boldsymbol{x},\ \boldsymbol{z}) = \begin{pmatrix} x_v + r\cos(\theta_v + \beta) \\ y_v + r\sin(\theta_v + \beta) \end{pmatrix}$$

- Since $\hat{\boldsymbol{x}}$ has a variable length we need to extend the state vector and the covariance matrix when we encounter a previously unseen landmark.
- The state vector is extended by the function $\boldsymbol{y}(\cdot)$:

$$\begin{aligned} \boldsymbol{x}'\langle k\rangle &= \boldsymbol{y}(\boldsymbol{x}\langle k\rangle,\ \boldsymbol{z}\langle k\rangle,\ \boldsymbol{x}_v\langle k\rangle) \\ &= \begin{pmatrix} \boldsymbol{x}\langle k\rangle \\ \boldsymbol{g}(\boldsymbol{x}_v\langle k\rangle,\ \boldsymbol{z}\langle k\rangle) \end{pmatrix} \end{aligned}$$

which appends the sensor-based estimate of the new landmark's coordinates to those already in the map.

DRAFT

- The covariance matrix also needs to be extended when a new landmark is observed, and this is achieved by

$$\hat{\mathbf{P}}'_{\langle k \rangle} = \mathbf{Y}_z \begin{pmatrix} \hat{\mathbf{P}}_{\langle k \rangle} & \mathbf{0}_{2\times M} \\ \mathbf{0}_{M\times 2} & \hat{\mathbf{W}} \end{pmatrix} \mathbf{Y}_z^\top$$

where M is the dimension of $\hat{\mathbf{P}}$ prior to it being extended; $\mathbf{Y}_z$ is the insertion Jacobian (A derivation can be found at sn.pub/XQv4jR):

$$\mathbf{Y}_z = \frac{\partial \boldsymbol{y}}{\partial \boldsymbol{z}} = \left(\begin{array}{cc|c} & \mathbf{1}_{M\times M} & \mathbf{0}_{M\times 2} \\ \mathbf{G}_x & \mathbf{0}_{2\times(M-3)} & \mathbf{G}_z \end{array} \right)$$

and the Jacobians are

$$\mathbf{G}_x = \frac{\partial \boldsymbol{g}}{\partial \boldsymbol{x}} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

$$\mathbf{G}_z = \frac{\partial \boldsymbol{g}}{\partial \boldsymbol{z}} = \begin{pmatrix} \cos(\theta_v + \beta) & -r\sin(\theta_v + \beta) \\ \sin(\theta_v + \beta) & r\cos(\theta_v + \beta) \end{pmatrix}$$

$\mathbf{G}_x$ is zero since $\boldsymbol{g}(\cdot)$ is independent of the map and therefore $\boldsymbol{x}$.

DRAFT

- An additional Jacobian for $\boldsymbol{h}(\cdot)$ is

$$\mathbf{H}_{p_i} = \frac{\partial \boldsymbol{h}}{\partial \boldsymbol{p_i}} = \begin{pmatrix} \frac{x_i - x_v}{r} & \frac{y_i - y_v}{r} \\ -\frac{y_i - y_v}{r^2} & \frac{x_i - x_v}{r^2} \end{pmatrix}$$

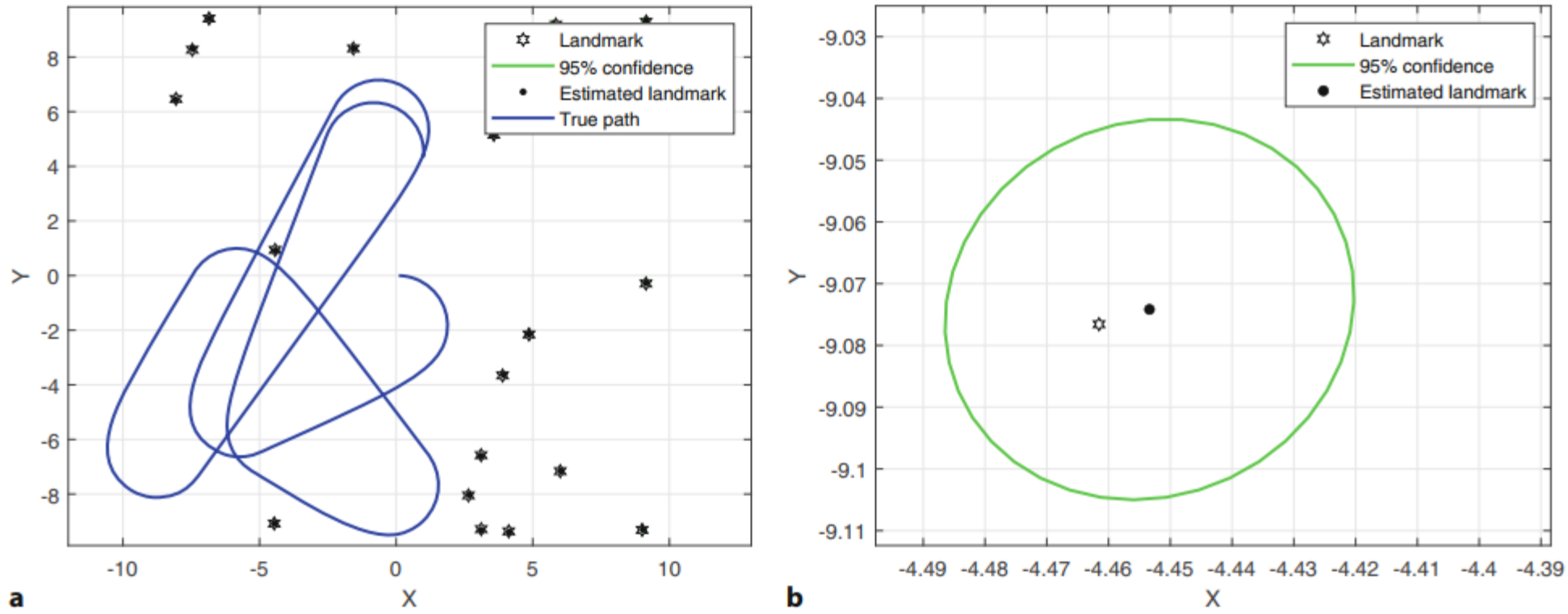
which describes how the sensor measurement changes with respect to the position of landmark i for a particular robot pose.

- For the mapping case, the Jacobian $\mathbf{H}_{\boldsymbol{x}}$ describes how the landmark observation changes with respect to the full state vector. However, the observation depends only on the position of the observed landmark, so this Jacobian is mostly zeros

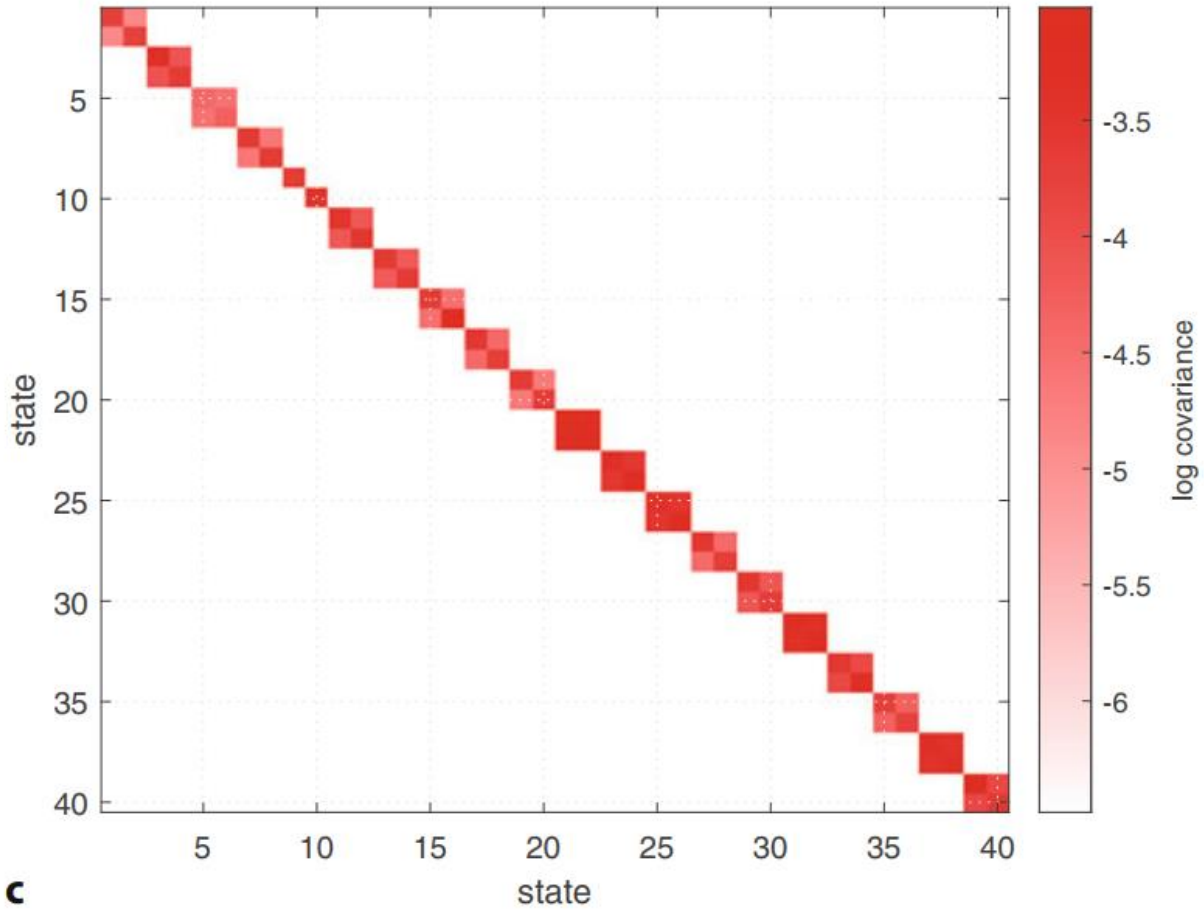
$$\mathbf{H}_x = \left.\frac{\partial \boldsymbol{h}}{\partial \boldsymbol{x}}\right|_{\boldsymbol{w}=0} = \left(\mathbf{0}_{2\times 2} \cdots \mathbf{H}_{p_i} \cdots \mathbf{0}_{2\times 2}\right) \in \mathbb{R}^{2\times 2M}$$

DRAFT

- Fig. a shows 95% confidence ellipses along with the true landmark positions and the path taken by the robot.
- Fig. b shows a close-up view of landmark 17 and we see that the estimate is close to the true value and well within the confidence ellipse.



- The covariance matrix is shown graphically in . Fig. c and has a blockdiagonal structure.
- The off-diagonal elements are zero, which implies that the landmark estimates are uncorrelated or independent.
- This is to be expected since observing one landmark provides no new information about any other landmark.



DRAFT

4.5.3.3 EKF SLAM

- Finally, we tackle the problem of determining our position and creating a map at the same time.
- This is an old problem in marine navigation and cartography – incrementally extending maps while also using the map for navigation.
- In robotics this problem is known as simultaneous localization and mapping (SLAM) or concurrent mapping and localization (CML).
- This is a "chicken and egg" problem since we need a map to localize, and we need to localize to make the map.

DRAFT

- The state vector comprises the robot configuration and the coordinates of the M landmarks that have been observed so far

$$\boldsymbol{x} = (x_v,\ y_v,\ \theta_v,\ x_1,\ y_1,\ x_2,\ y_2,\ \ldots\ x_M,\ y_M)^\top \in \mathbb{R}^{(2M+3)\times 1}$$

and therefore, has a variable length. The estimated covariance is a variable-sized symmetric matrix with a block structure

$$\hat{\mathbf{P}} = \begin{pmatrix} \hat{\mathbf{P}}_{vv} & \hat{\mathbf{P}}_{vm} \\ \hat{\mathbf{P}}_{vm}^\top & \hat{\mathbf{P}}_{mm} \end{pmatrix} \in \mathbb{R}^{(2M+3)\times(2M+3)}$$

where $\hat{\mathbf{P}}_{vv} \in \mathbb{R}^{3\times 3}$ is the covariance of the robot pose; $\hat{\mathbf{P}}_{mm} \in \mathbb{R}^{2M\times 2M}$ is the covariance of the estimated landmark positions; and $\hat{\mathbf{P}}_{vm} \in \mathbb{R}^{3\times 2M}$ is the correlation between robot and landmark states.

DRAFT

- When a new feature is observed, the state vector is updated using the insertion Jacobian Yz (in this case $\mathbf{G}_x$ is not zero)

$$\mathbf{G}_x = \frac{\partial \boldsymbol{g}}{\partial \boldsymbol{x}} = \begin{pmatrix} 1 & 0 & -r\sin(\theta_v + \beta) \\ 0 & 1 & r\cos(\theta_v + \beta) \end{pmatrix}$$

since the estimate of the new landmark depends on the state vector which now contains the robot's configuration.

- For the SLAM case, the Jacobian $\mathbf{H}_x$ describes how the landmark observation changes with respect to the state vector. The observation depends only on two elements of the state vector: the position of the robot and the position of the observed landmark

$$\mathbf{H}_x = \left.\frac{\partial \boldsymbol{h}}{\partial \boldsymbol{x}}\right|_{\boldsymbol{w}=0} = \left(\mathbf{H}_{x_v} \cdots \mathbf{0}_{2\times 2} \cdots \mathbf{H}_{p_i} \cdots \mathbf{0}_{2\times 2}\right) \in \mathbb{R}^{2\times(2M+3)}$$

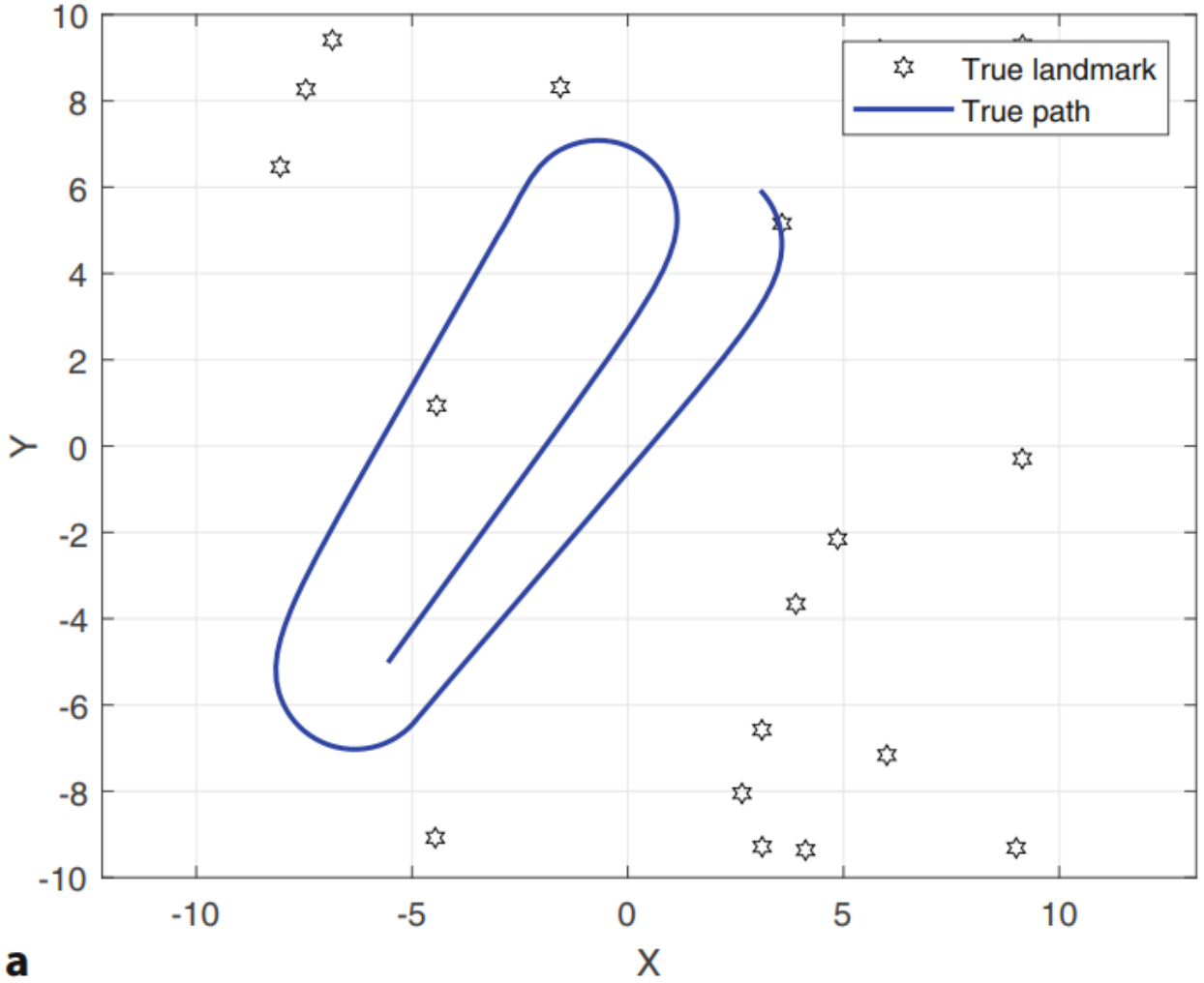
where $\mathbf{H}_{pi}$ is at the location in the state vector corresponding to landmark i.

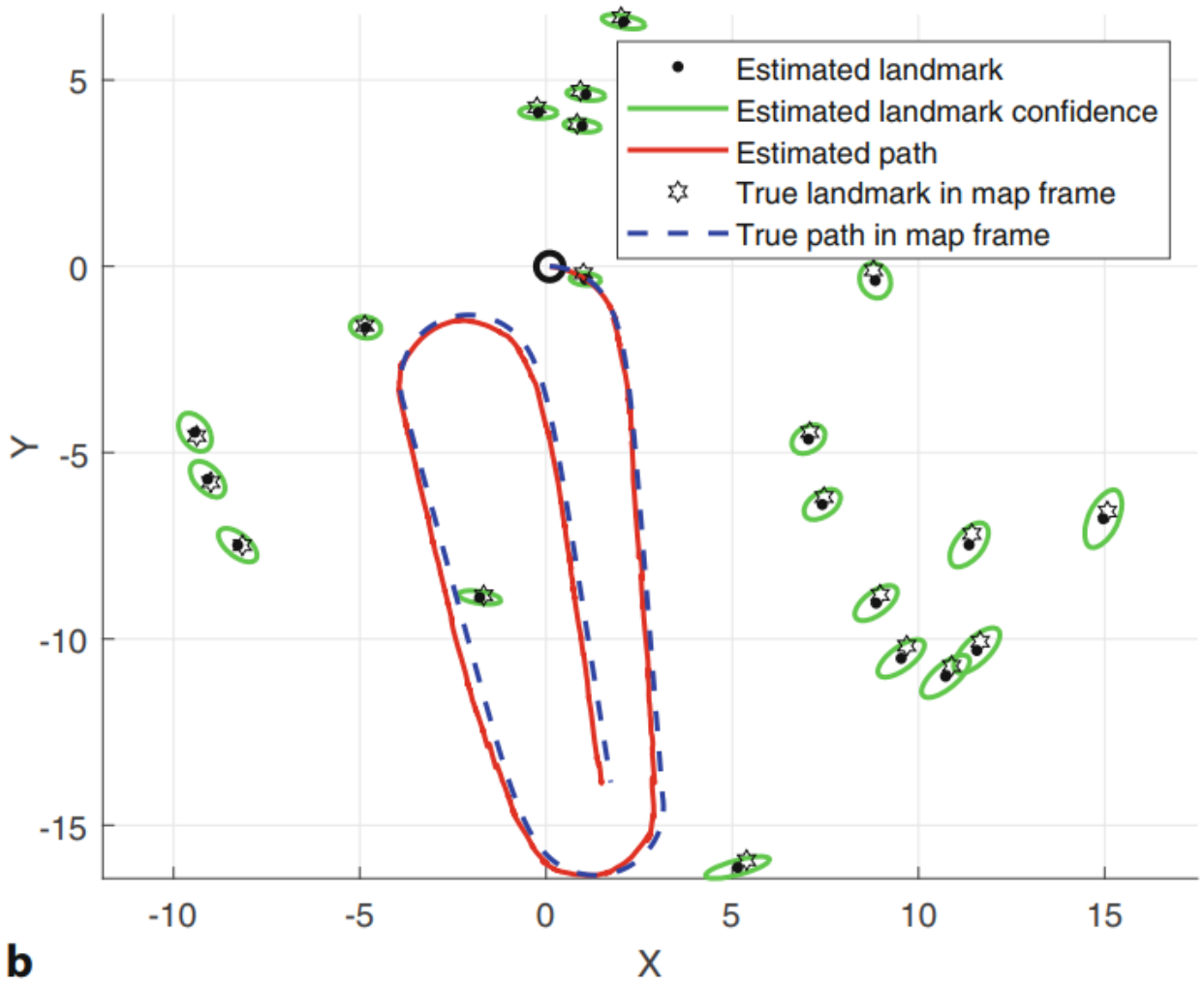
DRAFT

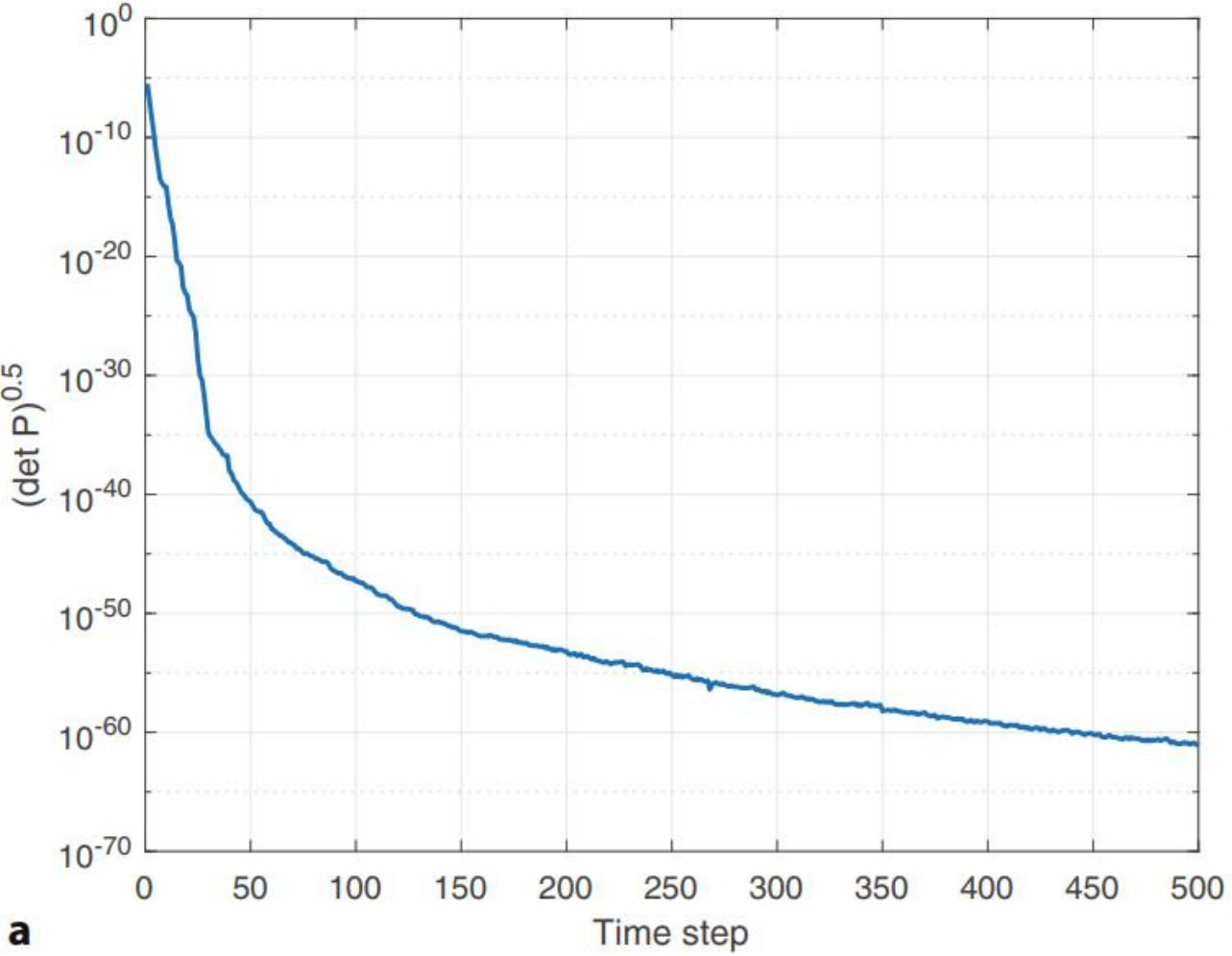
- The Kalman gain matrix K distributes innovation from the landmark observation, a 2-vector, to update every element of the state vector – the configuration of the robot and the position of every landmark in the map.

DRAFT

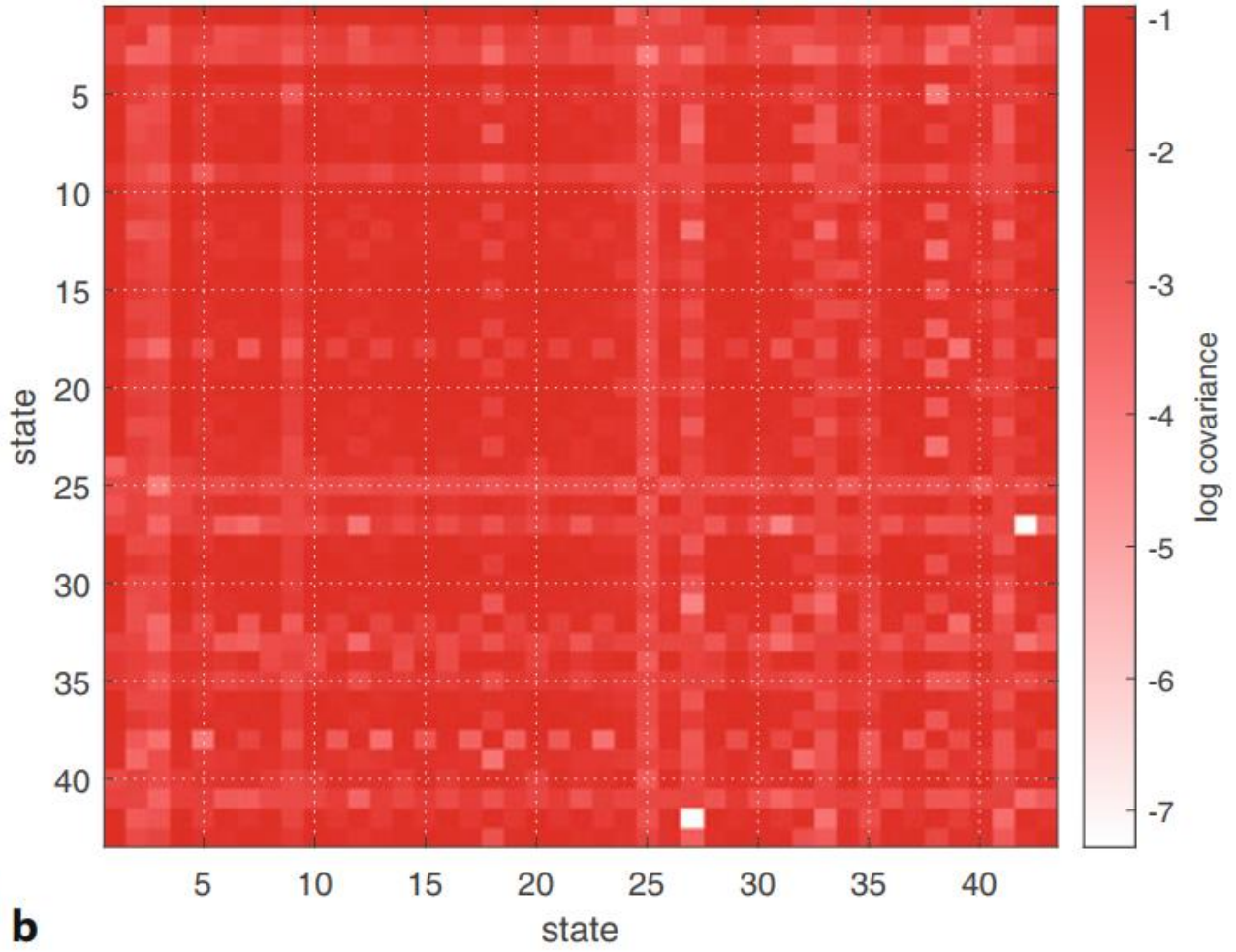
- The initial configuration of the robot is $(3{,}6, -45^{o})$ but the EKF is using the default robot configuration of $(0{,}0{,}0)$.






10^{0}
10^{-10}
10^{-20}
10^{-30}
10^{-40}
10^{-50}
10^{-60}
10^{-70}
(det P)$^{0.5}$
0
50
100
150
200
250
300
350
400
450
500
Time step

a


5
10
15
20
25
30
35
40
state
5
10
15
20
25
30
35
40
state
-1
-2
-3
-4
-5
-6
-7
log covariance

b

- $\widehat{\mathbf{P}}_{mm}$ is not block diagonal and the finite off-diagonal terms represent correlations between the landmarks in the map.
- The block $\widehat{\mathbf{P}}_{vm}$ is the correlation between errors in the robot configuration and the landmark positions.
- A landmark's pose estimate is a function of the robot's pose, so errors in the robot position appear as errors in the landmark position – and vice versa.
- The correlations are used by the Kalman filter to apply the innovation from a landmark observation to improve the estimate of every other landmark in the map as well as the robot configuration.
- Conceptually, it is as if all the states were connected by springs and the movement of any one affects all the others.

DRAFT

- The extended Kalman filter is a powerful tool for estimation but has a number of limitations.
- Firstly, the size of the matrices involved increases with the number of landmarks and can lead to memory and computational bottlenecks as well as numerical problems.
- Secondly, the underlying assumption of the Kalman filter is that all errors have a Gaussian distribution, and this is far from true for sensors like lidars that we will discuss later in this chapter.
- Thirdly, it requires good estimates of covariance of the noise sources, which in practice are challenging to estimate.

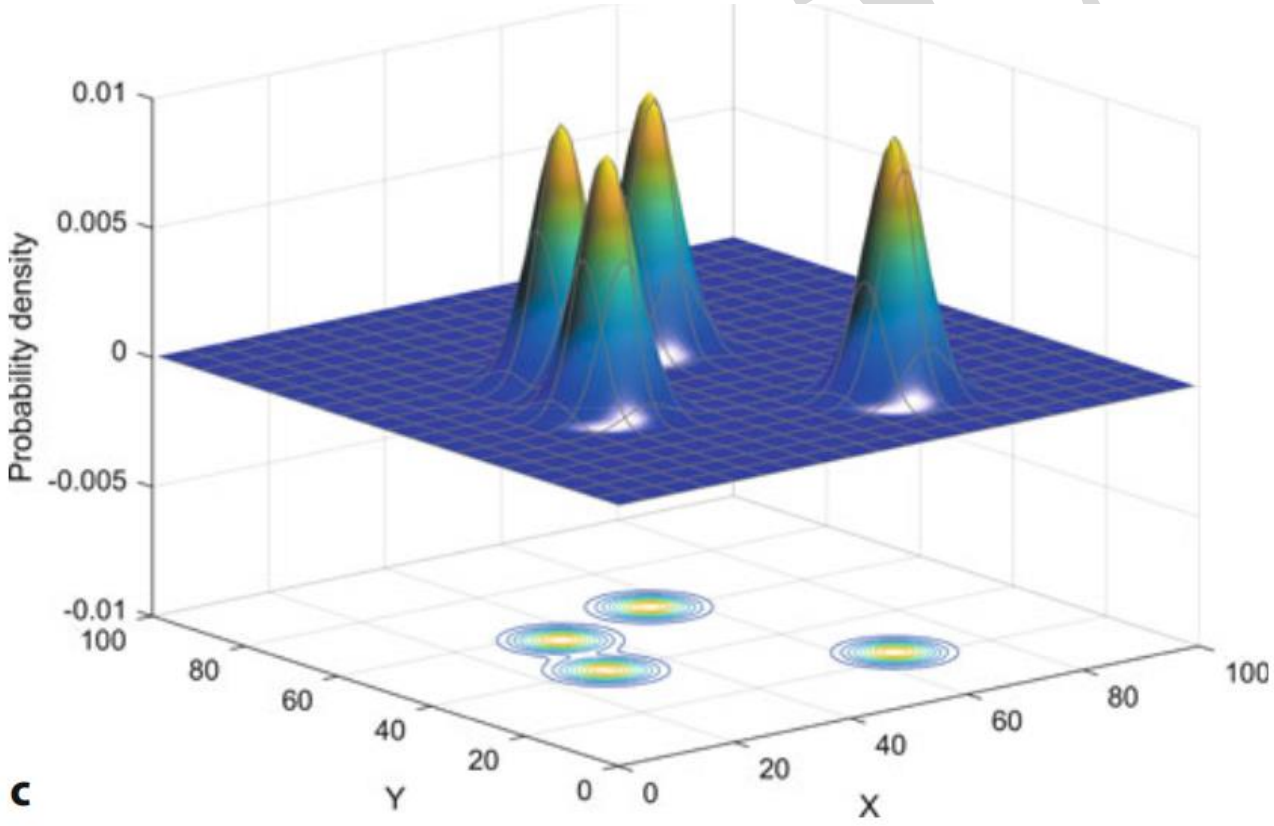
DRAFT

4.5.3.3 Sequential Monte-Carlo Localization

- The estimation examples so far have assumed that the errors in sensors, such as odometry and landmark range, have a Gaussian probability density function.
- In practice we might find that a sensor has a one-sided distribution (like a Poisson distribution) or a multi-modal distribution with several peaks.
- The functions we used in the Kalman filter are strongly nonlinear, which means that sensor noise with a Gaussian distribution will not result in a Gaussian error distribution on the value of the function.

DRAFT

- The probability density function associated with a robot's configuration may have multiple peaks to reflect several hypotheses that equally well explain the data from the sensors.



- The Monte-Carlo estimator that we discuss in this section makes no assumptions about the distribution of errors.
- It can also handle multiple hypotheses for the state of the system. The basic idea is disarmingly simple.
- We maintain many different values of the robot's configuration or state vector. When a new measurement is available, we score how well each particular value of the state explains what the sensor just observed.
- We keep the best fitting states and randomly sample from the prediction distribution to form a new generation of states.
- Collectively, these many possible states, and their scores, form a discrete approximation of the probability density function of the state we are trying to estimate.

DRAFT

- There is never any assumption about Gaussian distributions nor any need to linearize the system.
- While computationally expensive, it is quite feasible to use this technique with today's standard computers and it is well suited to parallel computation.
- If we plot these state vectors as points in the state space, we have a cloud of particles each representing a plausible robot state. Hence this type of estimator is often referred to as a particle filter.

DRAFT

- We will apply Monte-Carlo estimation to the problem of localization using odometry and a map.
- The estimator is initialized by creating N particles $\boldsymbol{x}_i$, $i = 1, \ldots, N$ distributed randomly over the configuration space of the robot.
- All particles have the same initial weight, importance, or likelihood: $w_i = 1/N$.
- The steps in the main iteration of the algorithm are:

1. Apply the state update to each particle

$$\boldsymbol{x}_i^+ \langle k+1 \rangle = \boldsymbol{f}(\boldsymbol{x}_i \langle k \rangle, \boldsymbol{u} \langle k \rangle + \boldsymbol{r} \langle k \rangle)$$

DRAFT

2. We make an observation $\boldsymbol{z}_j$ of landmark j that has, according to the map, coordinate $\boldsymbol{p}_j$. For each particle we compute the innovation

$$\boldsymbol{v}_i = \boldsymbol{h}\left(\boldsymbol{x}_i^+,\ \boldsymbol{p}_j\right) - \boldsymbol{z}_j$$

which is the error between the predicted and actual landmark observation.

A likelihood function provides a scalar measure of how well the particular particle explains this observation. In this example we choose a likelihood function

$$w_i = \mathrm{e}^{-\boldsymbol{v}_i^\top \mathbf{L}^{-1} \boldsymbol{v}_i} + w_0$$

to determine the weight of the particle, where L is a covariance-like matrix, and $w_0 > 0$ ensures that there is a finite probability of a particle being retained despite sensor error.

DRAFT

3. Select the particles that best explain the observation, a process known as resampling or importance sampling.

Given N particles $\boldsymbol{x_i}$ with corresponding weights w_i, we first normalize the weights

$$w_i' = \frac{w_i}{\sum_{i=1}^{N} w_i}$$

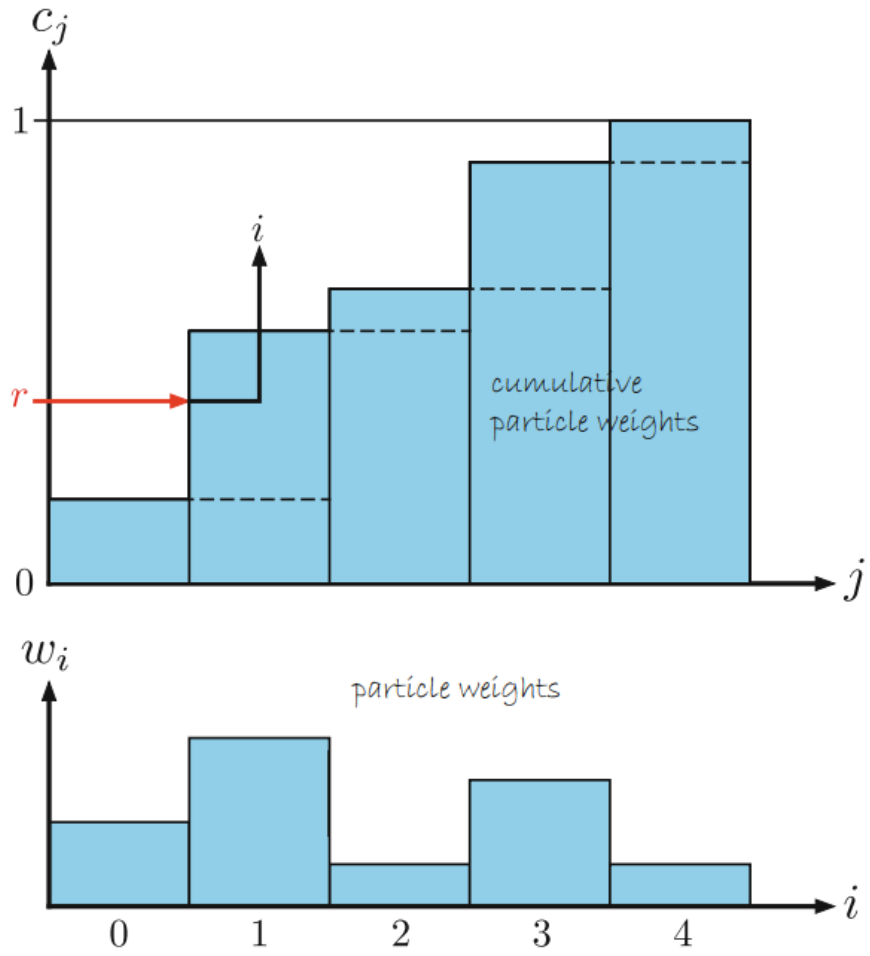
and construct a cumulative histogram

$$c_j = \Sigma_{i=1}^{j} w_i'$$

We then draw a uniform random number $r \in [0,1]$ and find

$$i = \arg \min_j r < c_j$$

and select particle i for the next generation.

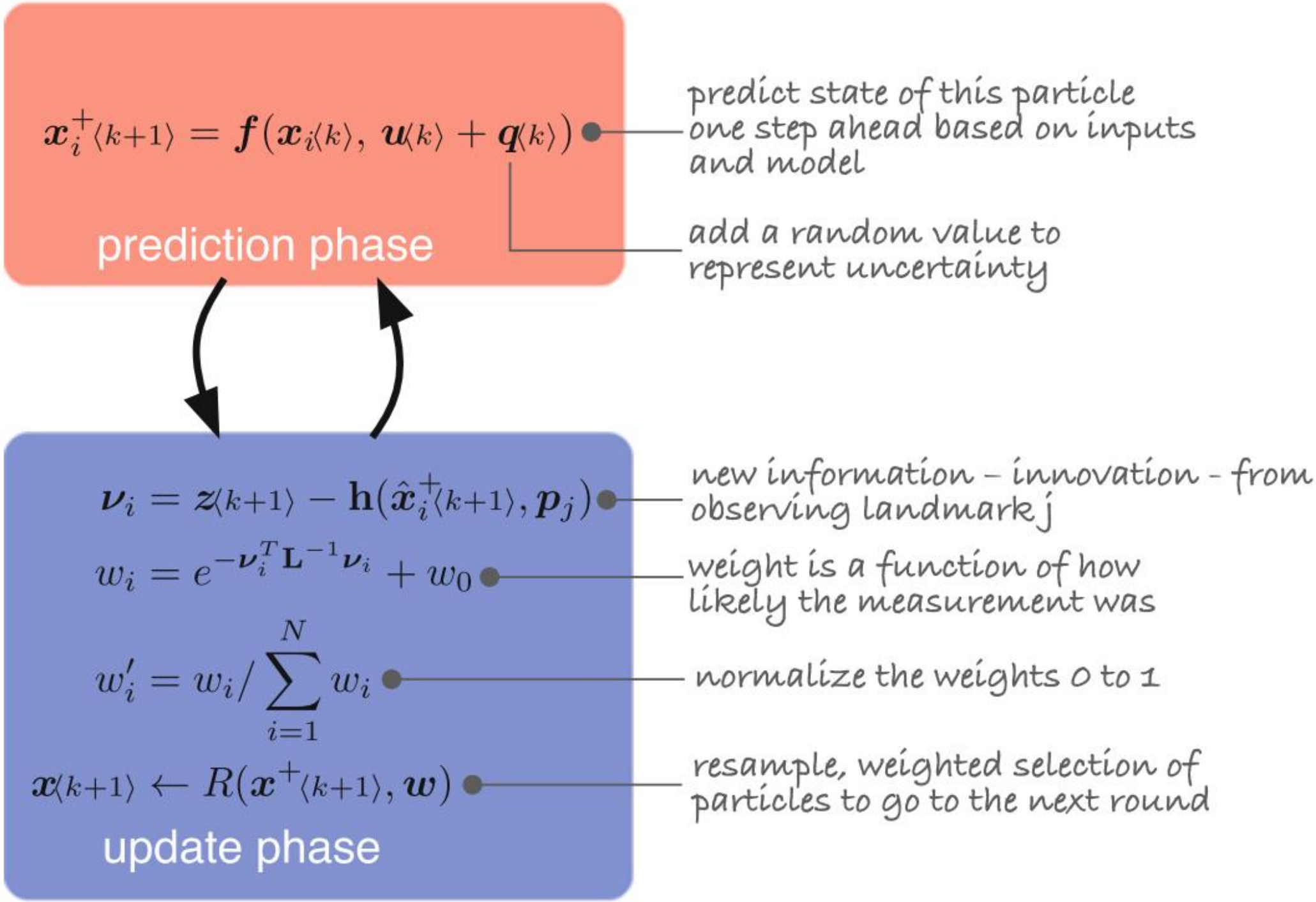


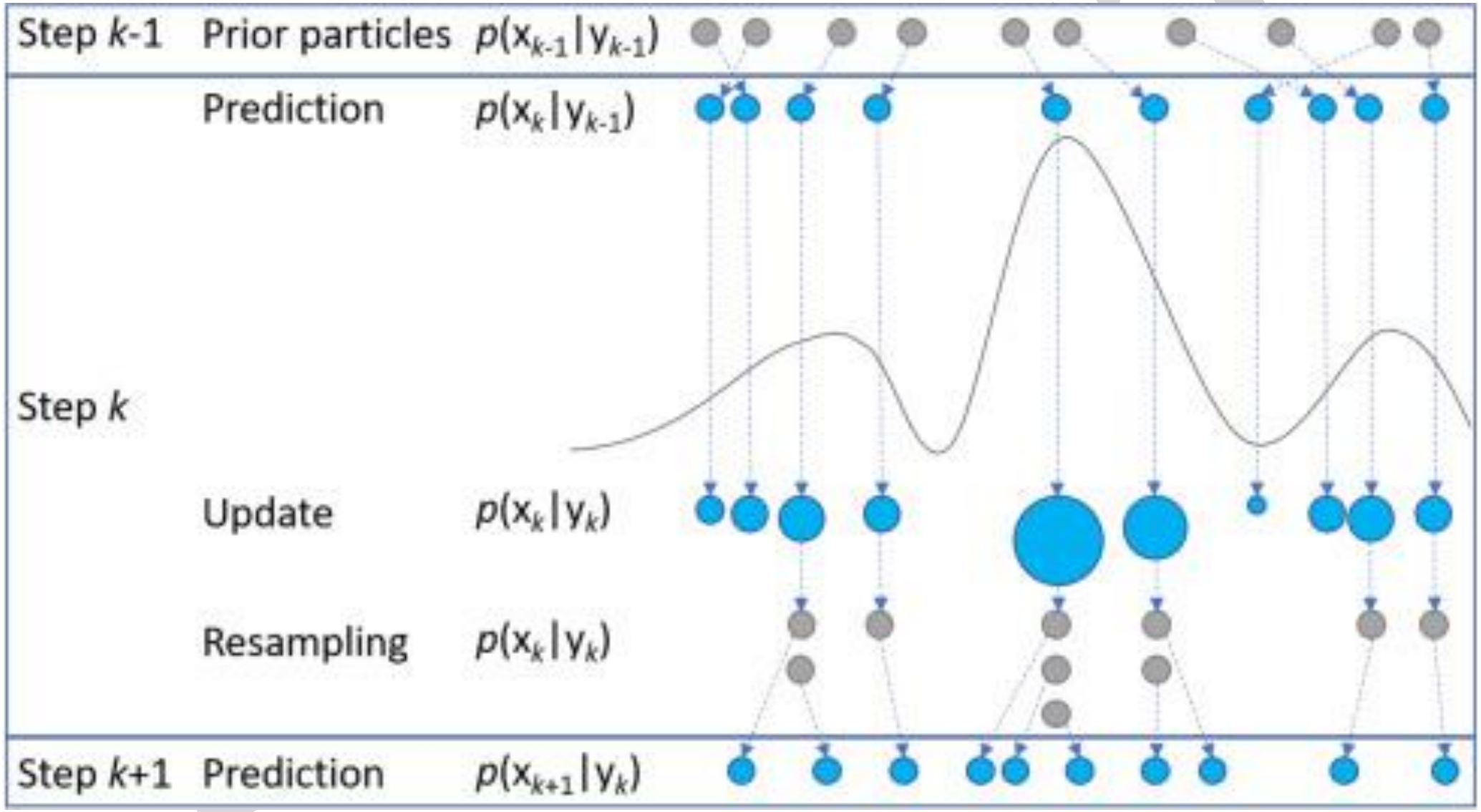
The probability of selecting a highly weighted particle like 1 or 3 is clearly higher than for a low-weighted particle like 2 or 4

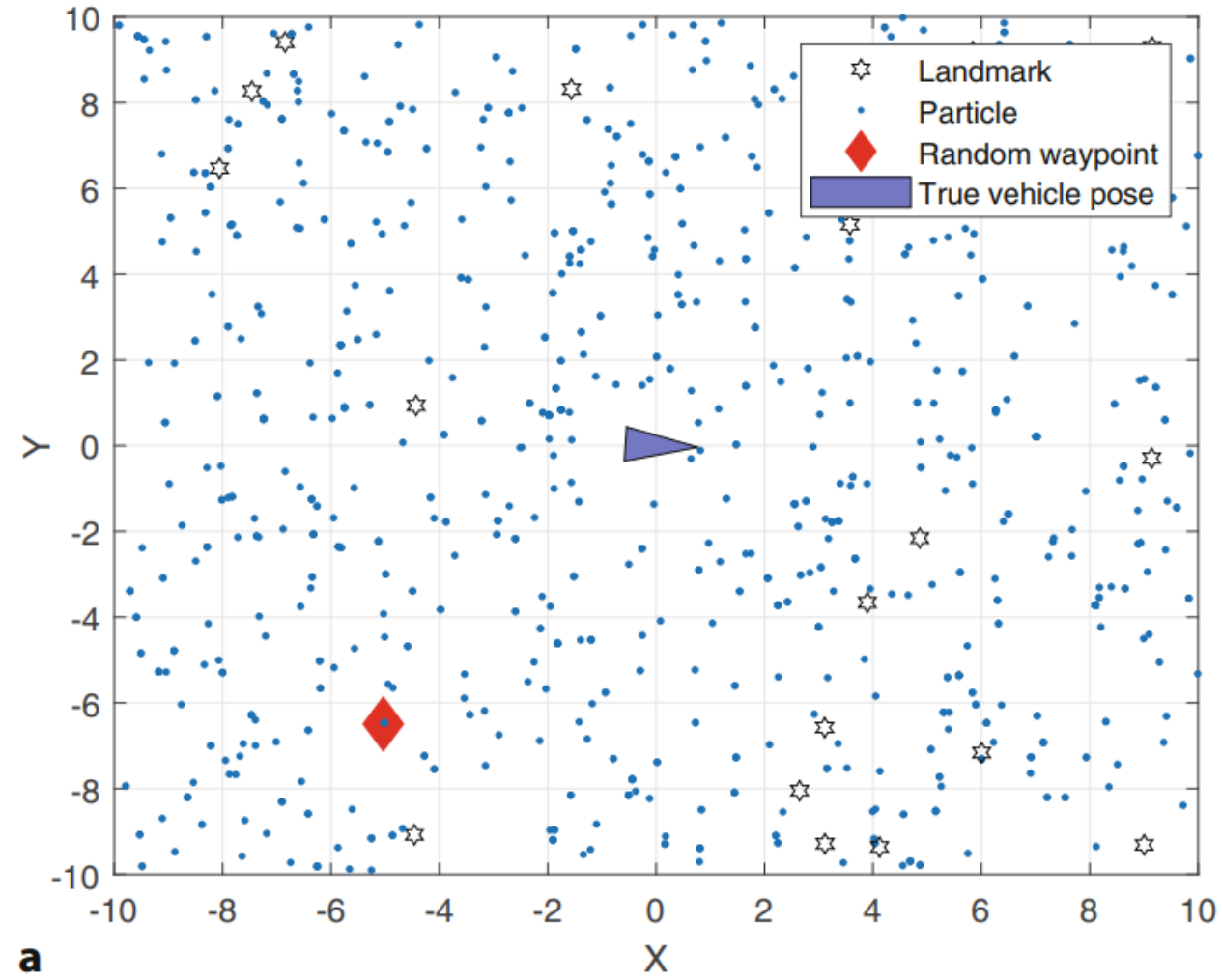
DRAFT

- The process is repeated N times.
- The end result will be a new set of N particles with multiple copies of highly weighted particles while some lower weighted particles may not be copied at all.
- Step 1 of the next iteration will spread out the identical particles through the addition of the random vector $\boldsymbol{r}\langle k \rangle$.
- This random spreading and resampling are critical components of the particle filter without which the filter would quickly produce a degenerate set of particles where a few have high weights and the bulk have almost zero weight which poorly represents the true distribution. This degenerate condition is also known as particle starvation.

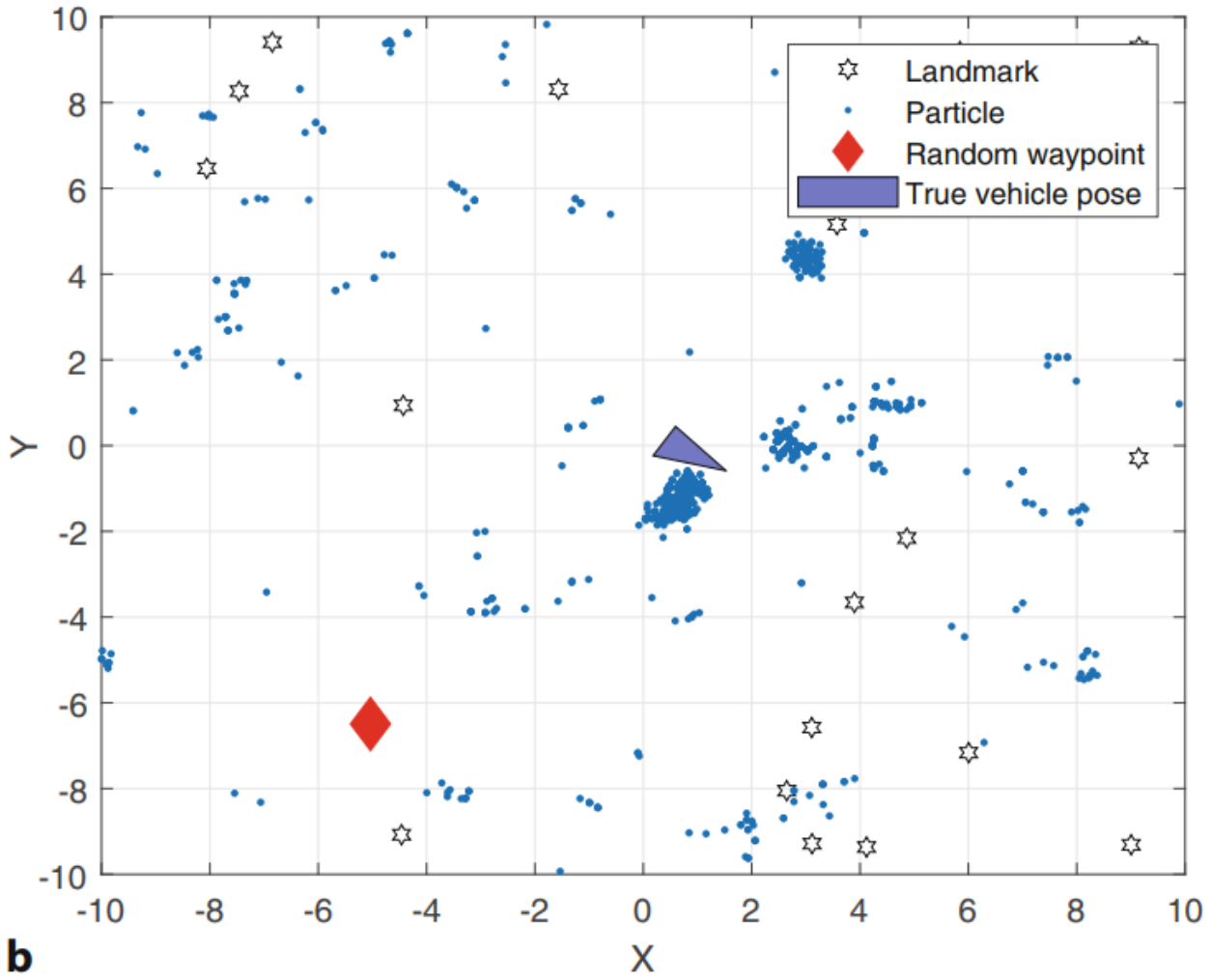
DRAFT


$\boldsymbol{x}_i^+\langle k+1\rangle = \boldsymbol{f}(\boldsymbol{x}_i\langle k\rangle, \boldsymbol{u}\langle k\rangle + \boldsymbol{q}\langle k\rangle)$
prediction phase
predict state of this particle one step ahead based on inputs and model
add a random value to represent uncertainty
$\boldsymbol{\nu}_i = \boldsymbol{z}\langle k+1\rangle - \mathbf{h}(\hat{\boldsymbol{x}}_i^+\langle k+1\rangle, \boldsymbol{p}_j)$
new information – innovation - from observing landmark j
$w_i = e^{-\boldsymbol{\nu}_i^T \mathbf{L}^{-1} \boldsymbol{\nu}_i} + w_0$
weight is a function of how likely the measurement was
$w_i' = w_i / \sum_{i=1}^{N} w_i$
normalize the weights 0 to 1
$\boldsymbol{x}\langle k+1\rangle \leftarrow R(\boldsymbol{x}^+\langle k+1\rangle, \boldsymbol{w})$
resample, weighted selection of particles to go to the next round
update phase

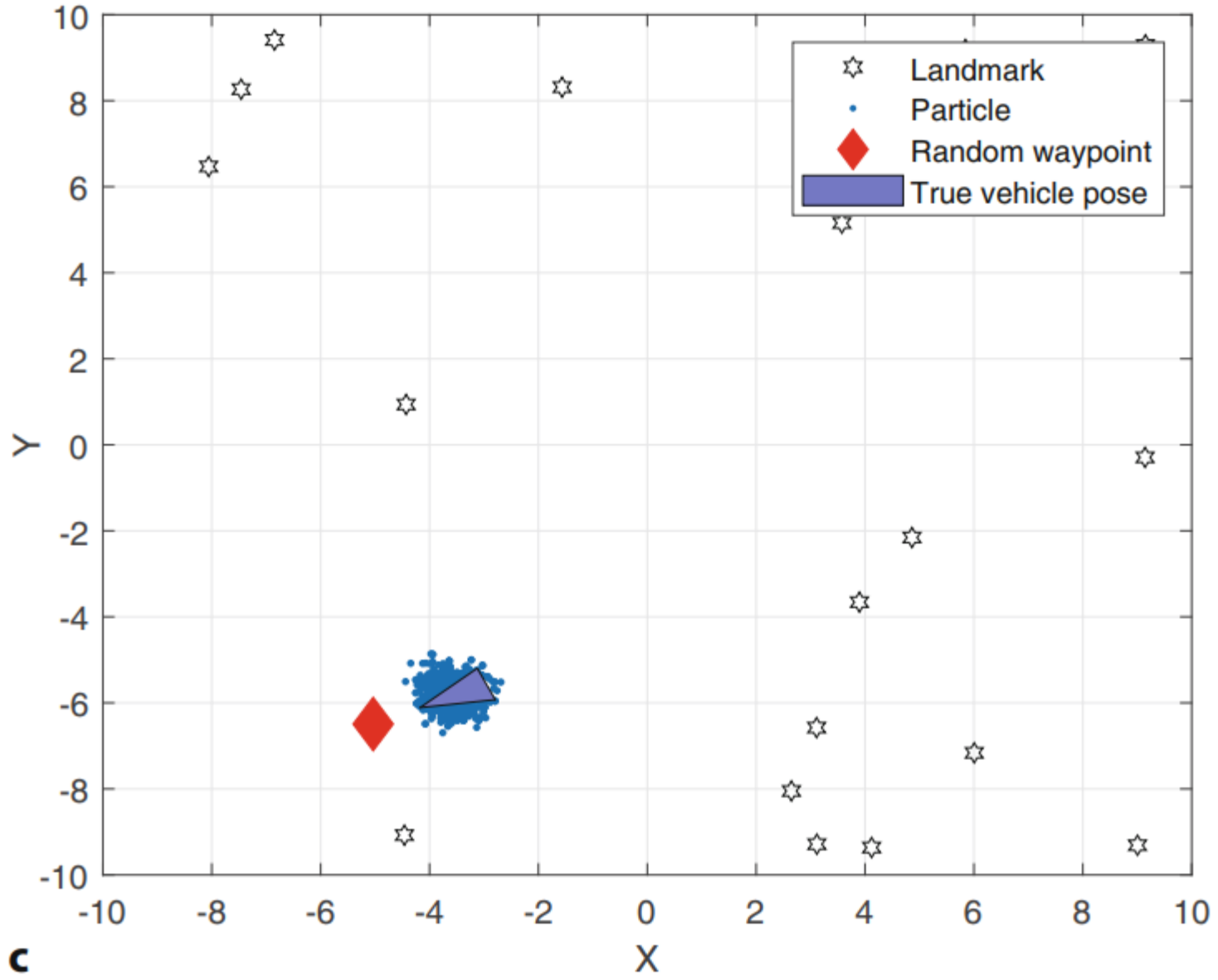

Step k-1
Prior particles
$p(x_{k-1}|y_{k-1})$
Prediction
$p(x_k|y_{k-1})$
Step k
Update
$p(x_k|y_k)$
Resampling
$p(x_k|y_k)$
Step k+1
Prediction
$p(x_{k+1}|y_k)$


Landmark
Particle
Random waypoint
True vehicle pose
X
Y
10
8
6
4
2
0
-2
-4
-6
-8
-10
-10
-8
-6
-4
-2
0
2
4
6
8
10

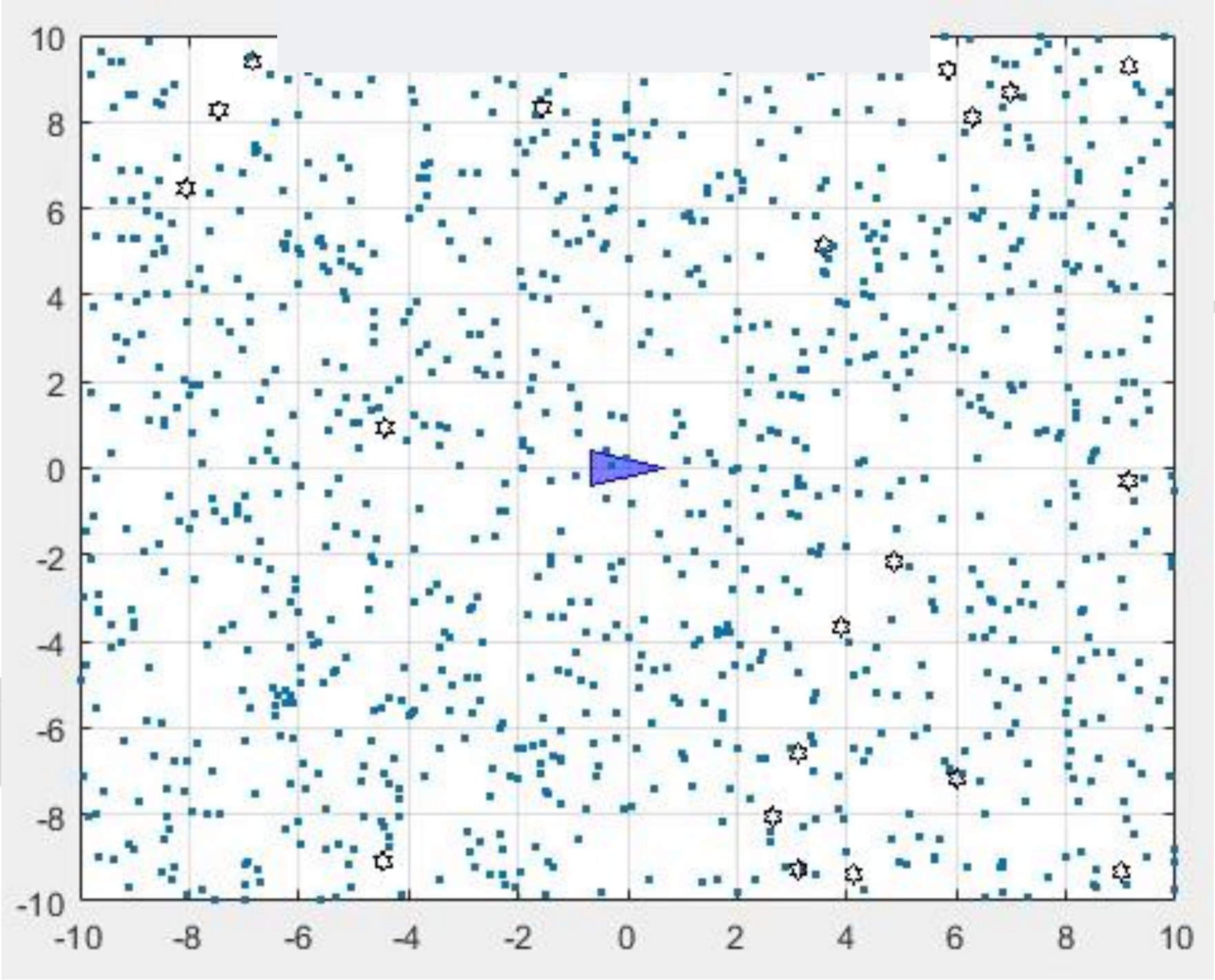
a


Landmark
Particle
Random waypoint
True vehicle pose
X
Y
10
8
6
4
2
0
-2
-4
-6
-8
-10
-10
-8
-6
-4
-2
0
2
4
6
8
10

b

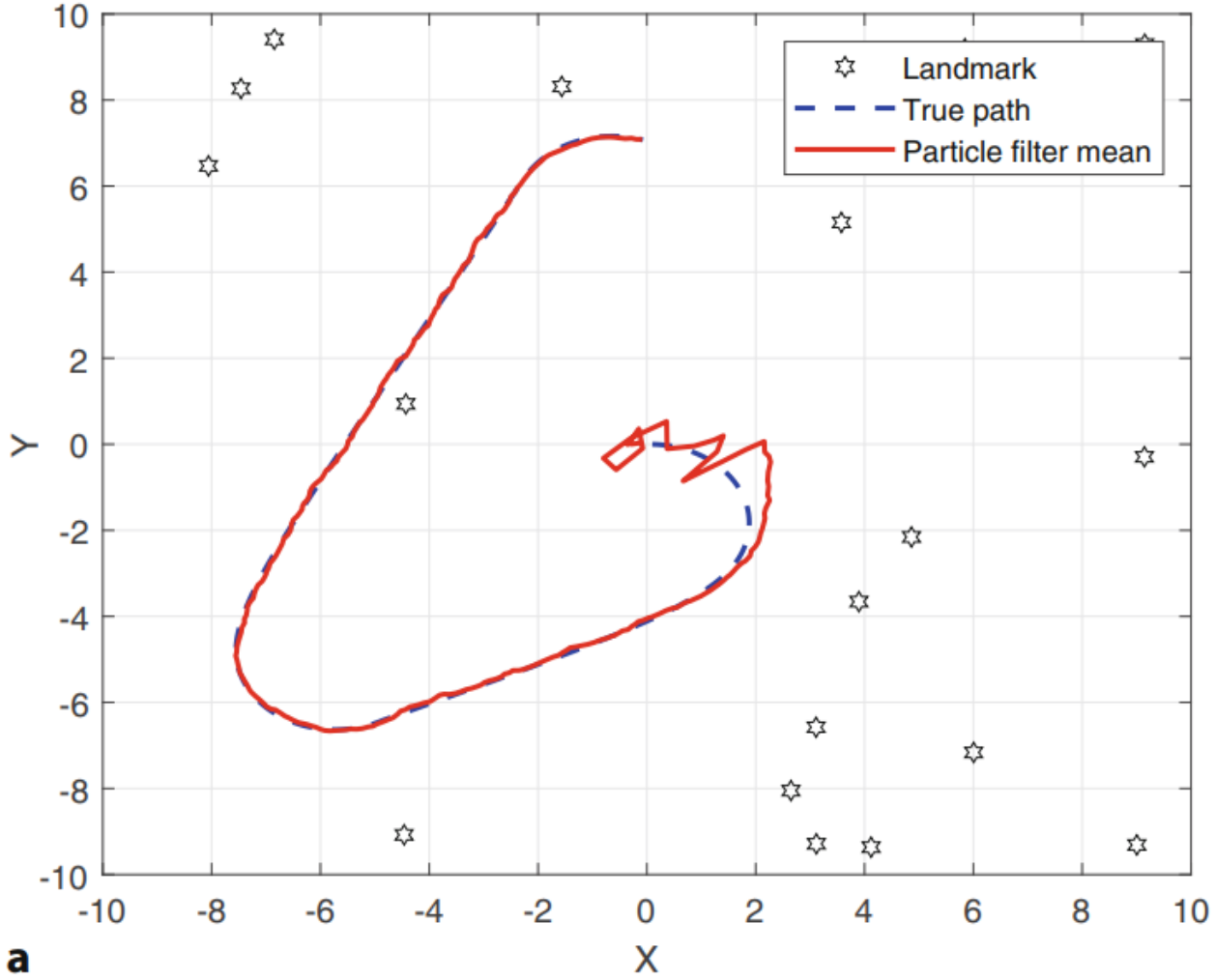

Landmark
Particle
Random waypoint
True vehicle pose
X
Y
10
8
6
4
2
0
-2
-4
-6
-8
-10
-10
-8
-6
-4
-2
0
2
4
6
8
10

c

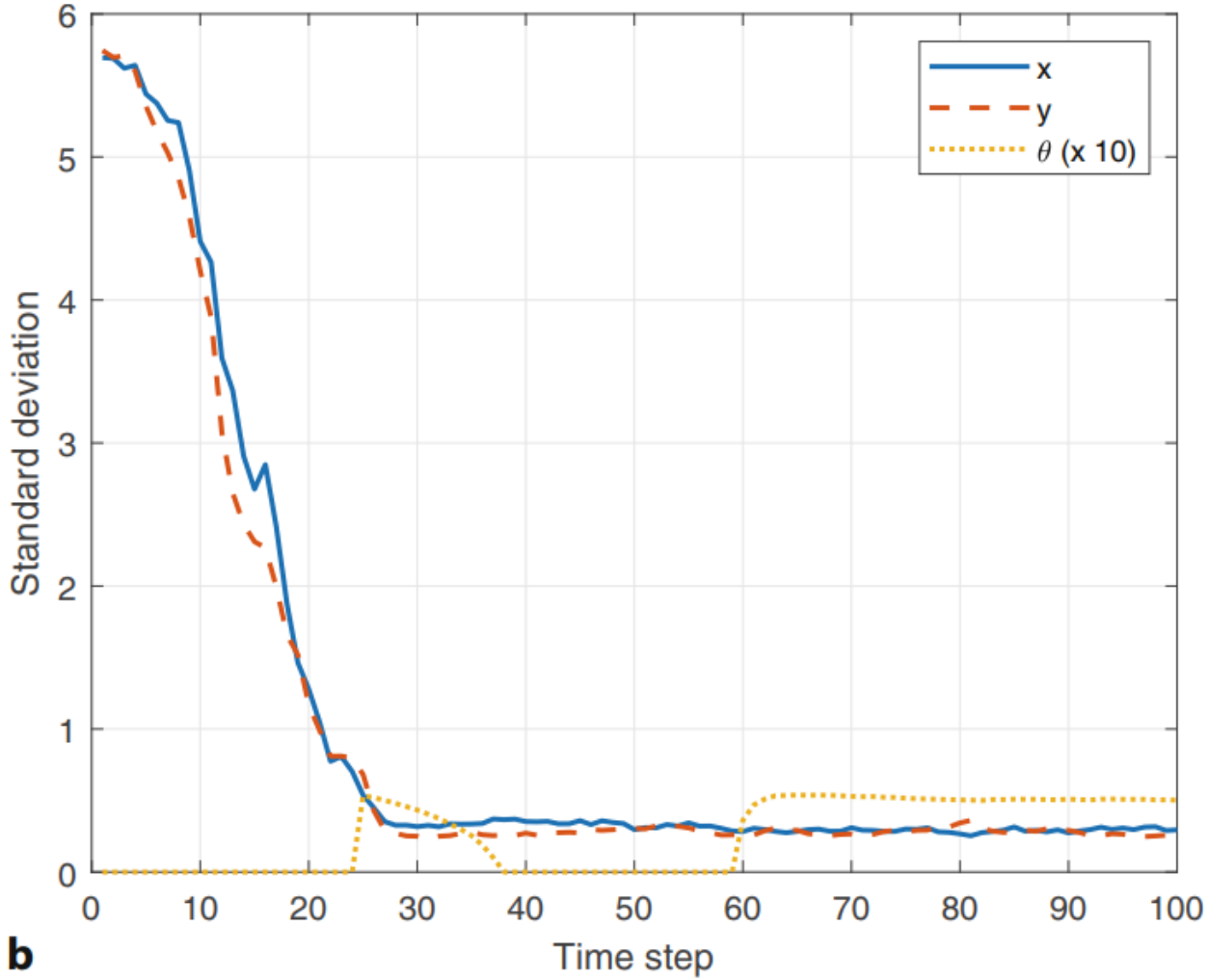

10
8
6
4
2
0
-2
-4
-6
-8
-10
-10
-8
-6
-4
-2
0
2
4
6
8
10

- We see the particles moving as their states are updated by odometry, random perturbation, and resampling. The initially randomly distributed particles begin to aggregate around those regions of the configuration space that best explain the sensor observations that are made.
- In Darwinian fashion, these particles become more highly weighted and survive the resampling step, while the lower-weighted particles are extinguished.
- The particles approximate the probability density function of the robot's configuration. The most likely configuration is the expected value or mean of all the particles.
- A measure of uncertainty of the estimate is the spread of the particle cloud or its standard deviation.

DRAFT


Landmark
True path
Particle filter mean
X
Y

a


x
y
θ (x 10)
Standard deviation
Time step

b

- The problem we have just solved is known in robotics as the kidnapped robot problem – a robot is placed in the world with no estimate of its initial pose and needs to localize itself as quickly as possible.
- To represent this large uncertainty, we uniformly distribute the particles over the 3-dimensional configuration space and their sparsity can cause the particle filter to take a long time to converge unless a very large number of particles is used.
- It is debatable whether this is a realistic problem. Typically, we have some approximate initial pose of the robot and the particles would be initialized to that part of the configuration space.
- For example, if we know the robot is in a corridor then the particles would be placed in those areas of the map that are corridors, or if we know the robot is pointing north then we would set all particles to have that orientation.

DRAFT

- Setting the parameters of the particle filter requires a little experience and the best way to learn is to experiment.
- For the kidnapped robot problem we set Q and the number of particles high so that the particles explore the configuration space but once the filter has converged, lower values could be used.
- There are many variations of the particle filter concerned with the shape of the likelihood function and the resampling strategy.
- The particle filter scales poorly to high-dimensional state spaces, since the number of particles required to approximate the state distribution quickly becomes intractable.

DRAFT

Additional resources

- https://www.mathworks.com/discovery/slam.html
- https://www.mathworks.com/help/nav/ug/ekf-based-landmark-slam.html
- https://www.mathworks.com/help/nav/ug/build-map-from-lidar-data-using-slam.html
- https://www.mathworks.com/help/nav/ug/map-indoor-area-using-lidar-slam-and-factor-graph.html
- https://www.mathworks.com/help/nav/ug/monocular-visual-simultaneous-localization-and-mapping.html

DRAFT